Stone IWC — Project Documentation

Source repository: https://github.com/stoneiwc/stoneiwc

Generated 2026-05-26 from `project-documentation/*.md`.

This bundle is a snapshot. The source files in the repo are the source of truth — if this PDF is older than a week, regenerate with `pnpm docs:pdf`.

> **For AI agents reading this PDF:** the full codebase is public at the URL above. Browse files directly (raw.githubusercontent.com works), or follow file references like `app/api/webhooks/stripe/route.ts` straight into the repo. The PDF describes the system; the repo IS the system.

Contents

Architecture — System Overview

What this application is

Stone International Wellness Center (Stone IWC) is a single-tenant Next.js 16 application that combines four business surfaces:

1. **Product storefront** — Sanity-managed products, cart, Stripe checkout, Sanity-mirrored orders, Resend confirmation emails.
2. **Service booking** — Cal.com-hosted bookings embedded under `/book/[slug]`.
3. **Editorial content** — Articles, team bios, hero slides, page images — all in Sanity.
4. **Contact** — Form submission via Resend, with auto-reply to the customer.

Everything is **one Vercel project**, deployed from this repo. There is no separate backend service.

High-level component map

The four integrations and what each owns

Integration	What it stores	What it does NOT store
Sanity	Products, categories, articles, team, hero slides, page images, **orders**, **gift cards**, coupons, shipping methods	Payment cards, transaction history (Stripe is the source of truth for the charge itself)
Stripe	PaymentIntents, charges, refunds, dispute records, customer payment methods	Order fulfillment status, customer addresses (carried only as PI metadata for admin reference)
Cal.com	Event types, bookings, calendars, scheduling pages	Anything about products or storefront orders
Resend	Email send logs (Resend dashboard), domain auth records	Templates (those live in this repo at `lib/email/`)

This separation matters: **Sanity is the canonical source for the order**, Stripe is the canonical source for the charge. The two are linked by `stripePaymentIntentId`.

End-to-end flows

Storefront purchase

```
1. Browser: customer adds items, applies coupon on /view-cart (cart context persisted to localStorage)
2. Browser: customer fills checkout form on /checkout, optionally applies gift card
3. Browser → POST /api/checkout/create-stripe-payment-intent
     ├── server creates Stripe PaymentIntent (amount = final discounted total in cents)
     ├── server enriches PI metadata: ship_*, items_summary, coupon_code, gift_card_code
     └── server creates Sanity `order` doc with status='pending'
4. Browser: Stripe Elements renders, customer confirms payment
5. Stripe → POST /api/webhooks/stripe (event: payment_intent.succeeded)
     ├── markOrderPaid → Sanity order status='paid', paidAt=now
     ├── if gift card: redeemGiftCard → Sanity balance updated
     └── Resend → order confirmation email to customer
6. Browser → /checkout/success?payment_intent=... (cart cleared)
```

Detailed: orders.md, checkout-and-payments.md.

Gift card purchase

A separate flow under `/products/stoneiwc-gift-certificate`. The product detail page renders a special purchase UI (gift-card.md). Gift cards do NOT create order documents — they get their own Sanity `giftCard` document on `payment_intent.succeeded`.

Service booking

```
1. Browser: customer browses /book or /book/[slug]
2. Page server-fetches the Cal.com event type (lib/cal-api.ts)
3. Page renders <CalBooker> which embeds Cal.com's hosted scheduling
4. Customer books inside the embed → Cal.com handles confirmation, calendar invites
   (the storefront has no further role)
```

Detailed: cal-com-booking.md.

Contact form

```
1. Browser → POST /api/contact (rate-limited by IP)
2. Server sends two emails via Resend:
   ├── Internal: to CONTACT_EMAIL_TO (info@), reply-to set to the customer
   └── Auto-reply: to the customer ("we received your message"), reply-to set to info@
```

Detailed: resend-email.md.

Data flow direction summary

```
Read (CDN-cached, ISR):
   Sanity ──► Next.js pages (revalidate 60s, tag-based invalidation)

Write — from customer browser:
   Browser ──► API route ──► Stripe (PaymentIntent.create)
                         ──► Sanity (createPendingOrder)
                         ──► Resend (contact form only)

Write — from Stripe (asynchronous):
   Stripe ──► /api/webhooks/stripe ──► Sanity (mark paid, redeem gift card)
                                   ──► Resend (order / gift card emails)

Write — from staff:
   Sanity Studio ──► Sanity dataset (products, articles, fulfillment updates)
```

The frontend **never writes to Sanity directly**. All writes go through API routes that hold the `SANITY_API_TOKEN` with Editor permission server-side.

Key conventions (used throughout the codebase)

Convention	Where
Prices are stored in **dollars** in Sanity and the cart, converted to **cents** only at the Stripe boundary (`Math.round(amount * 100)`)	`create-stripe-payment-intent/route.tsx`
Gift card product is identified by the slug `stoneiwc-gift-certificate` (single source of truth: `GIFT_CARD_SLUG`)	`components/products/gift-card-purchase.tsx`
Cal.com slugs ending in `-free` render as "Free", containing `-consultation-` show "Price for Consultation"	`cal-com-booking.md`
All ISR pages: `export const revalidate = 60` + Sanity queries tagged with their schema name	`sanity-cms.md`
Gift cards apply only to merchandise (capped at `subtotal - couponDiscount`); shipping is always paid → no $0 orders	`orders.md`
Webhooks are idempotent: order/gift card lookups happen by `stripePaymentIntentId` before any write	`webhooks.md`
`replyTo: CONTACT_EMAIL_TO` on all transactional emails to improve inbox placement	`resend-email.md`
Product / coupon / shipping data is server-fetched via API routes (never client-direct to Sanity) to keep dataset auth private	`sanity-cms.md`

Where things live (file layout)

```
app/
├── api/                          Server routes (Stripe, Sanity writes, contact)
│   ├── checkout/                 Storefront payment intent + status helpers
│   ├── gift-cards/               Gift card PI creation + validation
│   ├── webhooks/stripe/          Single Stripe webhook handler (both events + gift cards)
│   ├── contact/                  Contact form + auto-reply
│   ├── validate-coupon/          Coupon code check (server-only Sanity access)
│   ├── shipping-methods/         Active shipping options
│   └── revalidate/               On-demand ISR revalidation hook
├── products/                     Listing + detail pages (SSG + ISR)
├── checkout/                     Cart checkout + success page
├── book/                         Service booking pages (Cal.com embed)
├── view-cart/                    Standalone cart page (coupon application lives here)
├── education/                    Articles + certifications
├── about/  featured/  services/  Editorial pages
├── contact/                      Contact form
├── sitemap.ts  robots.ts         SEO surface
└── layout.tsx                    Root layout (CartProvider lives here)

components/
├── checkout/                     SelfCheckoutSection, CheckoutDetailsSection
├── products/                     Listing grid, filters, gift-card-purchase
├── home/  education/  contact/   Page-specific composites
├── seo/                          JSON-LD components
├── ui/                           shadcn/ui primitives
├── cart-sheet.tsx                Slide-over cart
├── cal-booker.tsx                Cal.com embed wrapper
├── footer.tsx  navbar.tsx        Layout chrome
└── ...

lib/
├── sanity.client.ts              Sanity client (write-capable on server)
├── sanity.queries.ts             All GROQ queries + TypeScript types
├── sanity.image.ts               urlFor() image builder
├── cart-context.tsx              Cart state + localStorage persistence
├── orders.ts                     createPendingOrder, markOrderPaid, markOrderFailed
├── gift-cards.ts                 createGiftCard, getGiftCardByCode, redeemGiftCard
├── cal-api.ts                    Cal.com REST client
├── products.ts                   Product type definitions (frontend shape)
├── navigation.ts                 Header/footer link config
├── email/                        Resend templates + rate limiter
└── utils.ts  constants.ts

studio/
├── sanity.config.ts              Studio structure (custom sidebar tree)
└── schemaTypes/                  Document schemas (product, order, giftCard, ...)

scripts/
└── cal_events_*.py               Cal.com event provisioning (Python, runs locally)

project-documentation/            This documentation
public/                           Static assets, favicons, OG images
```

Environments (development → production)

Branch	Vercel scope	Domain	Stripe mode	Purpose
`development`	Preview (aliased)	`dev.stoneiwc.com`	Test (`sk_test_…`)	Integration / staging
`production`	Production	`stoneiwc.com`	Live (`sk_live_…`)	Customer-facing
feature branch	Preview (auto URL)	`…vercel.app`	Test	PR previews

Vercel env vars are scoped: each variable can hold different values per scope. See environments-and-deployments.md for the full matrix and setup steps.

What this architecture deliberately does NOT do (yet)

- **No customer accounts.** Orders are tied to email, not user IDs. Order history page would require auth (planned, not built).
- **No abandoned-cart cleanup.** Pending orders sit in Sanity forever unless an admin deletes them, or a future Vercel Cron sweeps them.
- **No server-side price re-validation.** The server trusts client-sent item prices. A determined attacker could send a low `totalAmount`. (Mitigated only by Sanity being read-only client-side and Stripe enforcing the charge amount exactly.)
- **No inventory tracking.** Sanity has `inStock` boolean per product but no quantity counter, no decrement on purchase.
- **No multi-currency.** USD only, hardcoded.
- **No analytics SDK.** Only Vercel's built-in deployment metrics.

These are documented in open-backlog.md so successors know what's intentional vs. unfinished.

Where to go next

If you want to…	Read
Set up the project locally	onboarding.md
Understand deploy + env config	environments-and-deployments.md
Debug a production issue	runbook.md
Add a product, service, or image	content-workflows.md
Modify the checkout flow	checkout-and-payments.md + orders.md
Add a new page or component	frontend-ui.md
Debug a webhook	webhooks.md

Onboarding — First Day Setup

A walk-through for a new engineer joining the project. By the end of this you should have the storefront running locally with hot reload, the Studio open in another tab, and a Stripe test purchase going end to end.

Estimated time: **45–60 minutes** if you have access to all the dashboards listed below.

Prerequisites

Tool	Version	Why
Node.js	20.x or 22.x LTS	Next.js 16 runtime
pnpm	9.x or newer	Package manager (lockfile is `pnpm-lock.yaml`)
Git	any	
A code editor	VS Code recommended	The repo has no editor lock-in
Stripe CLI (optional)	latest	For local webhook forwarding
Python 3.11+ (optional)	for `scripts/`	Only needed if you'll provision Cal.com events

You will also need accounts (or invites) to:

- **Sanity** — the project's dataset
- **Stripe** — at least Test mode access
- **Resend** — send domain access (or just an API key)
- **Cal.com** — the team account (`stoneiwc` username)
- **Vercel** — the project (for env vars + deploys)

Ask the maintainer for invites before starting.

---

1. Clone & install

```
git clone https://github.com/stoneiwc/stoneiwc.git
cd stoneiwc
pnpm install
```

The Studio has its own `package.json`. Install it too:

```
cd studio
pnpm install
cd ..
```

---

2. Environment variables

Copy the template and fill it in:

```
cp .env.template .env.local
```

Then populate each variable. Sources:

Variable	Where to get it
`NEXT_PUBLIC_SANITY_PROJECT_ID`	Sanity Manage → Project → API → Project ID
`NEXT_PUBLIC_SANITY_DATASET`	`production` (the only dataset)
`NEXT_PUBLIC_SANITY_API_VERSION`	A date string like `2024-10-01`. Match the one in `lib/sanity.client.ts`
`SANITY_API_TOKEN`	Sanity Manage → Project → API → Tokens → create one with **Editor** permission
`STRIPE_SECRET_KEY`	Stripe Dashboard → **Test mode** → Developers → API keys → `sk_test_…`
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Same screen, `pk_test_…`
`STRIPE_WEBHOOK_SECRET`	See "Stripe webhook (local)" section below
`RESEND_API_KEY`	Resend Dashboard → API Keys
`RESEND_FROM_EMAIL`	`info@stoneiwc.com` (must be a verified domain)
`CONTACT_EMAIL_TO`	`info@stoneiwc.com`
`NEXT_PUBLIC_CAL_USERNAME`	`stoneiwc`
`CAL_API_KEY`	Cal.com → Settings → Developer → API keys
`NEXT_PUBLIC_FRONTEND_URL`	`http://localhost:3000` for local

The Studio also needs its own copy of the Sanity project ID + dataset. Create `studio/.env`:

```
SANITY_STUDIO_PROJECT_ID=your_project_id
SANITY_STUDIO_DATASET=production
```

(Same values as the root, no token.)

---

3. Run the app

In two terminals:

```
# Terminal A — the storefront
pnpm dev
# → http://localhost:3000

# Terminal B — the Studio
cd studio && pnpm dev
# → http://localhost:3333
```

You should see:

- `localhost:3000` renders the homepage with hero, featured products, etc. Content comes from Sanity.
- `localhost:3333` shows the Studio with the structured sidebar (Pages → Home / About / …, Products, Gift Cards, Orders, etc.).

If the storefront renders an empty page or 500s, the most common cause is missing/wrong Sanity env vars. Check the terminal for the actual error.

4. Stripe webhook (local)

The webhook handler at `/api/webhooks/stripe` is required for orders to flip from `pending` to `paid` after checkout, and for gift card balance updates. To exercise this locally:

Option A — Stripe CLI (recommended)

Install the Stripe CLI (stripe.com/docs/stripe-cli), then:

```
stripe login
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

The CLI prints a webhook signing secret (`whsec_…`). Copy that into your `.env.local` :

```
STRIPE_WEBHOOK_SECRET=whsec_...
```

Restart `pnpm dev` . Now every Stripe event in your account will be relayed to local.

Option B — Skip webhooks locally

If you only need to test checkout UI without the backend side effects (Sanity order, emails), you can omit `STRIPE_WEBHOOK_SECRET` . The PaymentIntent will still create and Stripe will still charge the test card — but Sanity won't get the `paid` status update and no email will go out. The route will throw on missing secret if the webhook is hit, but Stripe can't reach `localhost` without forwarding anyway.

---

5. Make a test purchase

1. Open `localhost:3000/products` — pick any product, add to cart.
2. Click cart icon → "View Cart" → optionally apply a coupon (Sanity Studio → Coupons → check what codes exist).
3. Click "Checkout" → fill the form → "Continue to Payment".
4. Card: `4242 4242 4242 4242` , any future expiry, any CVC, any ZIP.
5. After success, verify:
   - Studio → Orders shows a new doc with status `paid`
   - Stripe Dashboard → Test mode → Payments shows the PaymentIntent
   - Your email (the one you typed) received the order confirmation (sent via Resend)

If status stays `pending` , the webhook didn't reach you — check Terminal B (`stripe listen` output) and Vercel function logs.

Detailed test scenarios: runbook.md and the gift card flow in gift-card.md.

---

6. Make a test booking

1. Open `localhost:3000/book` — see the service list (comes from Cal.com via `lib/cal-api.ts`).
2. Click a service. The Cal.com booking embed should appear under `/book/[slug]` .
3. You can book a slot — it'll appear in the team's Cal.com calendar.

If the embed shows blank, check `NEXT_PUBLIC_CAL_USERNAME` and `CAL_API_KEY` in `.env.local` .

---

7. Common project conventions to know

These are the things you'll trip over in week 1 if you don't know them:

Convention	Source
Prices in dollars everywhere, converted to cents only at the Stripe API call	`create-stripe-payment-intent/route.tsx`
The gift card is a Sanity product with the slug `stoneiwc-gift-certificate`. Don't rename it.	`components/products/gift-card-purchase.tsx`
Cart state is in React context + localStorage, no server session.	`lib/cart-context.tsx`
All Sanity reads are server-side through `lib/sanity.queries.ts`. No client `useEffect` fetch from Sanity.	
ISR with `revalidate = 60` on most pages — your changes in Studio take up to a minute to appear (or trigger `/api/revalidate`).	
Studio structure is manual — adding a schema requires registering in 3 places. See `sanity-cms.md`.	
Webhook is idempotent — re-delivering a Stripe event doesn't double-redeem a gift card.	
Email send failures are logged but don't break checkout — customer always sees success after payment, even if Resend hiccups.	

8. Git workflow

Branch	Purpose
`production`	Production — deploys to `stoneiwc.com`
`development`	Staging — deploys to `dev.stoneiwc.com`
feature branches	PR previews (auto Vercel URL)

Typical flow:

```
git checkout development
git pull
git checkout -b feat/my-thing
# … work …
git push -u origin feat/my-thing
gh pr create --base development
```

Production: merge `development` → `production` only after `dev.stoneiwc.com` smoke test. See environments-and-deployments.md.

Commit style (observed from history): Conventional commits — `feat(scope):`, `fix(scope):`, `docs(scope):`. Body explains the "why", wrapped at ~72 chars. No "Co-Authored-By" trailer.

9. Where to keep digging

You need to…	Read
Understand the system at a high level	architecture.md
Deploy, manage env vars, separate test vs live	environments-and-deployments.md
Add a product, service, or image	content-workflows.md
Fix a broken order / spam email / failed webhook	runbook.md
Touch the checkout or payment code	checkout-and-payments.md, orders.md, gift-card.md
Touch the UI	frontend-ui.md
Understand the webhook handler	webhooks.md

Have a question that's not in the docs? Add it to the docs after you find the answer. Successors will thank you.

Environments & Deployments

How the project moves from a local laptop to `dev.stoneiwc.com` and finally to `stoneiwc.com`, and how every secret is scoped along the way.

Branches → environments

There are three environments. They differ in **Vercel scope**, **Stripe mode**, and **public URL**. The repo is connected to GitHub; pushes auto-deploy.

Vercel scope	Git source	URL	Stripe	Purpose
Production	`production` branch	`https://stoneiwc.com`	Live (`sk_live_…`)	Real customers, real money
Preview	`development` branch (aliased) + any PR branch	`https://dev.stoneiwc.com` (alias) or auto-generated `*.vercel.app`	Test (`sk_test_…`)	Staging + per-PR previews
Development	None (Vercel scope used by `vercel dev` locally)	`localhost:3000`	Test	Used rarely; `vercel dev` reads this scope when needed

> `dev.stoneiwc.com` is a **custom domain alias** for the latest Preview deployment off the `development` branch. It always points to the most recent push on that branch. PR branches get their own `*.vercel.app` preview but don't take over `dev.stoneiwc.com`.

The env var matrix

Every variable below has a **scope**. Same key, different value per scope is supported and expected for the Stripe + URL variables. To split a single existing variable into per-scope values, see "Splitting an existing variable" below.

Sanity — same across scopes

Variable	Production	Preview	Development	Notes
`NEXT_PUBLIC_SANITY_PROJECT_ID`	same	same	same	Public, fine to share
`NEXT_PUBLIC_SANITY_DATASET`	`production`	`production`	`production`	We use one dataset for all environments
`NEXT_PUBLIC_SANITY_API_VERSION`	same	same	same	A date string
`SANITY_API_TOKEN`	Editor token	Editor token	Editor token	Same token; **Editor** permission required for order + gift card writes

> Why one dataset across all scopes: test orders in Sanity are easy to filter out (status, prefix, etc.) and there's no need to maintain a separate Studio. If this becomes painful, create a `development` Sanity dataset and split here.

Stripe — MUST differ between Production and Preview

Variable	Production	Preview	Development
`STRIPE_SECRET_KEY`	`sk_live_…`	`sk_test_…`	`sk_test_…`
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	`pk_live_…`	`pk_test_…`	`pk_test_…`
`STRIPE_WEBHOOK_SECRET`	Live webhook's `whsec_…`	Test webhook's `whsec_…`	Local `stripe listen` `whsec_…`

> Critical: if Production accidentally has `sk_test_…`, real customer cards won't be charged and the live Stripe Dashboard won't show anything. Always verify with `vercel env pull --environment=production` before announcing a release.

Resend — same across scopes (currently)

Variable	Production	Preview	Development	Notes
`RESEND_API_KEY`	same	same	same	One key, sender domain is verified
`RESEND_FROM_EMAIL`	`info@stoneiwc.com` (or `Stone IWC <info@stoneiwc.com>`)	same	same	Including a display name improves inbox placement
`CONTACT_EMAIL_TO`	`info@stoneiwc.com`	same	same	Where contact form submissions land + the reply-to of transactional emails

Cal.com — same across scopes

Variable	All scopes
`NEXT_PUBLIC_CAL_USERNAME`	`stoneiwc`
`CAL_API_KEY`	The team API key

App — must differ

Variable	Production	Preview	Development
`NEXT_PUBLIC_FRONTEND_URL`	`https://stoneiwc.com`	`https://dev.stoneiwc.com`	`http://localhost:3000`

This is used in some email templates and metadata. Wrong value → emails link to the wrong host.

Optional

Variable	Notes
`SANITY_WEBHOOK_SECRET`	If `/api/revalidate` is wired to Sanity webhooks; otherwise unused

Splitting an existing variable across scopes

If a variable is currently set to "All Environments" but you need a different value per scope (typical for `STRIPE_*`), here's the safe sequence in the Vercel dashboard (Project → Settings → Environment Variables):

1. **Edit the existing entry.** Uncheck **Production** so it covers only Preview + Development. Save.
2. **Add a new entry** with the same key (`STRIPE_SECRET_KEY` etc.), the production-only value, and only **Production** checked. Save.
3. Verify with `vercel env ls` — you should see the same key listed twice with different scope columns.
4. **Redeploy** both Production and the latest Preview so the new values get picked up. Updated env vars do **not** auto-apply to existing deployments.

Symptoms of forgetting step 4: the deployment was built with the old env vars, so even though the dashboard shows the new value, your traffic still uses the old one. Always trigger a fresh deploy.

Stripe webhooks (one per mode)

Two webhook endpoints exist in Stripe — one in Test mode, one in Live mode. Each has its own signing secret (`whsec_…`).

Mode	URL	Events	Used by
Test	`https://dev.stoneiwc.com/api/webhooks/stripe?x-vercel-protection-bypass=…&x-vercel-set-bypass-cookie=true`	`payment_intent.succeeded`, `payment_intent.payment_failed`	Preview deployments
Live	`https://stoneiwc.com/api/webhooks/stripe`	Same	Production

> The `?x-vercel-protection-bypass=…` query string is only needed in Preview because **Vercel Deployment Protection** is enabled there by default — Stripe would otherwise hit a Vercel SSO/password challenge and get a 401. Production is public, so no bypass param is needed (or wanted — leaving it in production is a needless secret leak).

How to get the bypass token: Vercel → Project → Settings → Deployment Protection → "Protection Bypass for Automation" → generate. Append `&x-vercel-set-bypass-cookie=true` so subsequent requests in the same session also bypass.

If Production deployments are getting 401 from Stripe, double-check that Deployment Protection is **disabled** for the Production scope (it should be by default).

Production merge checklist

Before merging `development` → `production`, walk through this list. Skipping any item has bitten us before.

Pre-merge

- [ ] All test scenarios in runbook.md pass on `dev.stoneiwc.com`
- [ ] Vercel env vars — confirm split per the matrix above. Particularly:
  - [ ] `STRIPE_SECRET_KEY` Production = `sk_live_…`
  - [ ] `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` Production = `pk_live_…`
  - [ ] `STRIPE_WEBHOOK_SECRET` Production = Live webhook's secret
  - [ ] `NEXT_PUBLIC_FRONTEND_URL` Production = `https://stoneiwc.com`
- [ ] Stripe Dashboard → Live mode → Webhooks → `https://stoneiwc.com/api/webhooks/stripe` listening to both events
- [ ] Sanity Studio deployed (`cd studio && npx sanity deploy`) so admins see latest schemas
- [ ] Resend domain `stoneiwc.com` Verified — SPF / DKIM / Return-Path / DMARC all green

Merge

```
git checkout main
git pull
git merge development --no-ff
git push origin main
```

Vercel auto-deploys. Watch the Production deploy in the Vercel dashboard. Wait for "Ready".

Post-merge smoke test (~5 minutes)

On `stoneiwc.com`:

- [ ] Real card, **smallest possible amount** (a $1 product or refund-after). Complete checkout.
- [ ] Sanity → Orders shows the new doc with status `paid`
- [ ] Stripe Dashboard → **Live mode** → Payments shows the charge
- [ ] Customer email inbox got the order confirmation
- [ ] Issue a **refund** in Stripe Dashboard for the test order, then set status → `refunded` in Sanity

If anything fails: don't panic. Check runbook.md for triage. The Sanity order is recoverable from Stripe metadata; the only unrecoverable failure is a live key issue, which the pre-merge checklist should have caught.

Rolling back

If a production deploy is bad, Vercel has instant rollback:

1. Vercel → Project → Deployments → find the last good production deploy
2. … menu → "Promote to Production"

This swaps the alias `stoneiwc.com` to the older build in seconds. No git revert needed for an emergency rollback (do that afterwards, on your own time).

Caveats:

- Rollback doesn't undo Sanity writes. If the bad deploy created broken order documents, you'll have to clean them up in Studio.
- Rollback doesn't change env vars. If the bad deploy was due to a wrong env var, fix the env var THEN redeploy. A rollback to an older deploy with the same wrong env var won't help.

CI / GitHub Actions

The repo currently has no GitHub Actions. Vercel handles build + deploy on push. There are no required status checks before merge — relying on PR review and the `development` smoke test.

If a CI pipeline is desired later: a typical addition is `pnpm lint` + `npx tsc --noEmit` on every push. The build itself runs on Vercel and surfaces failures in the PR's "Checks" tab.

Local dev with `vercel dev`

You normally use `pnpm dev` for local development. `vercel dev` is only needed if you want the Vercel routing layer (rewrites, headers from `vercel.json`) — there's currently nothing in `vercel.json` that diverges from Next.js's own behavior, so `pnpm dev` is fine.

If you do use `vercel dev`:

```
vercel link    # one-time, links this repo to the Vercel project
vercel env pull .env.development   # pulls Development scope env vars
vercel dev     # runs on port 3000 by default
```

Where each piece lives

What	Where
Env vars	Vercel → Project → Settings → Environment Variables
Build logs	Vercel → Project → Deployments → click a deployment → "Building" tab
Runtime function logs	Vercel → Project → Logs (or click into a specific deployment)
Stripe events	Stripe Dashboard → Developers → Events / Webhooks
Sanity data	`https://your-studio-host.sanity.studio` (hosted Studio) or `localhost:3333` locally
Resend send logs	Resend Dashboard → Logs
Cal.com bookings	Cal.com → Bookings

See also

- runbook.md — when a deploy is broken, look here first
- webhooks.md — webhook URL / secret details
- security-and-compliance.md — env var hygiene, secret rotation

Runbook — Troubleshooting Playbook

Common failure modes, how to diagnose them, and how to fix them. Skim the symptoms first; click into the section that matches what you're seeing.

> Triage order when something is broken in production:
>
> 1. **Vercel function logs** (Vercel → Logs) — most issues surface here
> 2. **Stripe Dashboard → Events** (filter to the affected PaymentIntent) — for any payment-adjacent issue
> 3. **Sanity Vision** (Studio → Vision tab) — to query orders / gift cards directly with GROQ
> 4. **Resend Logs** (Resend Dashboard → Logs) — for email problems

---

Incidents

Payment succeeded but no Sanity order

Symptoms: Customer sees the success page. Stripe shows a charge. Sanity Studio → Orders has nothing for that PaymentIntent.

Diagnosis:

1. Find the PaymentIntent ID in Stripe → Payments → click the payment → copy `pi_…`
2. Search Vercel function logs for that ID. Look for the line `Sanity order creation failed for PaymentIntent pi_…`
3. If found, the error after it tells you why (token permission, schema validation, network).

Fix:

- **Recover the missing order:** the Stripe PaymentIntent has all the data you need in `metadata` (ship_*, items_summary, customer_email, totals). Manually create a Sanity Order document in Studio with the same fields. Set `stripePaymentIntentId` and `status: 'paid'` so future webhook retries are idempotent.
- **Fix the root cause:** usually `SANITY_API_TOKEN` lacks Editor permission (see Sanity write fails).

Why this doesn't break the customer experience: `createPendingOrder` is wrapped in try/catch in `create-stripe-payment-intent/route.tsx`. A Sanity failure logs but doesn't break the PaymentIntent return. The customer still pays; the

admin loses visibility.

Order stuck in "pending" forever

Symptoms: Sanity Order has status `pending` even though Stripe shows the payment as succeeded (often >1 minute later).

Diagnosis:

1. Stripe → Developers → Webhooks → click the endpoint → "Event deliveries"
2. Find the `payment_intent.succeeded` event for this PaymentIntent
3. Look at the response: is it 200? Or did Stripe never deliver?

Common causes:

- Webhook delivery failed (401, 500, timeout). See specific 401/400 sections below.
- Webhook returned 200 but our handler logged an error in `markOrderPaid`. Check Vercel logs.
- Order document exists but webhook's `stripePaymentIntentId` lookup is missing because the PI was created in test mode and the order was created in production (env var mismatch).

Fix:

- **Manual replay:** Stripe → click the failed event → "Resend". The handler is idempotent, so it's safe.
- **Manual flip:** Studio → open the order → change status to `paid` and stamp `paidAt`. Document the reason in `internalNotes`.

Stripe webhook returns 401

Symptoms: Stripe Event delivery shows `401 Unauthorized` response.

Cause: Vercel Deployment Protection is enabled for the deployment Stripe is hitting (typically a Preview), and the webhook URL doesn't include the bypass token.

Fix:

- Preview/dev: the webhook URL in Stripe Test mode must be `https://dev.stoneiwc.com/api/webhooks/stripe?x-vercel-protection-bypass=…&x-vercel-set-bypass-cookie=true`. Regenerate the bypass token in Vercel → Settings → Deployment Protection if needed.
- Production: ensure Deployment Protection is **disabled** for Production scope. Webhook URL must be the clean `https://stoneiwc.com/api/webhooks/stripe` (no query string).

See environments-and-deployments.md.

Stripe webhook returns 400 — signature verification failed

Symptoms: Vercel function log: `Webhook signature verification failed: …`. Stripe shows 400.

Cause: `STRIPE_WEBHOOK_SECRET` doesn't match the secret of the webhook endpoint that's sending events.

This usually happens because:

- The Vercel env value is from the Test mode endpoint but Stripe is sending from the Live mode endpoint (or vice versa)
- The webhook secret was rotated in Stripe and not updated in Vercel
- The wrong env scope was edited

Fix:

1. Stripe Dashboard → Developers → Webhooks → click the endpoint → "Signing secret" → reveal
2. Vercel → Settings → Environment Variables → confirm `STRIPE_WEBHOOK_SECRET` for the matching scope contains the same value
3. After updating, redeploy (env vars don't apply to existing deployments)

Gift card not redeemed after successful purchase

Symptoms: Customer used a gift card. Order completed. But Sanity → Gift Cards shows the balance unchanged.

Diagnosis:

1. Vercel logs for `Gift card redemption failed:`
2. Stripe PaymentIntent metadata should have `gift_card_code` and `gift_card_applied_amount`. If absent, the redeem path was skipped.
3. Sanity → Gift Cards → search by code → check the `redemptions` array

Common causes:

- Webhook didn't receive the event (see stuck pending)
- `SANITY_API_TOKEN` lacks Editor permission
- The gift card was deactivated between checkout and webhook arrival
- The card hit the optimistic-concurrency retry limit (5 attempts) — high contention, very rare

Fix:

- Manually adjust `currentBalance` and append to `redemptions` in Studio
- Then fix the root cause as above

Gift card balance double-decremented

Symptoms: Balance dropped by 2× the expected amount after a single order.

Cause: Two webhook deliveries for the same event reached the handler, AND the idempotency check failed. Should be impossible given the current code, but if you observe it, do this:

Fix:

1. Open the gift card doc in Studio
2. The `redemptions` array tells you exactly what was applied. If two entries have the same `stripePaymentIntentId`, that's the bug — adjust `currentBalance` upwards by the duplicate amount and delete one of the entries
3. File a bug — the idempotency guard in `lib/gift-cards.ts` `redeemGiftCard` needs review

---

Email goes to spam / Promotions / Junk

Symptoms: Customers report they don't see the order/gift card email in their inbox. Found in Spam or Gmail's Promotions tab.

Diagnosis & checks (in order of likely impact):

1. **Resend domain auth:** Resend Dashboard → Domains → `stoneiwc.com` → all of SPF, DKIM, Return-Path, DMARC must be **Verified**. If any are missing/red, add the DNS records to the domain registrar.

2. **DMARC policy:** start with `p=none` (monitoring only). After 1–2 weeks of clean reports, can advance to `p=quarantine`.
3. **From address:** `RESEND_FROM_EMAIL` should ideally include a display name: `Stone International Wellness Center <info@stoneiwc.com>` instead of just `info@stoneiwc.com`. Display names reduce spam scores.
4. **Reply-to:** All transactional emails have `replyTo: CONTACT_EMAIL_TO` set so the message looks like two-way mail, not a no-reply.
5. **Subject lines:** Avoid `$` amounts, ALL CAPS, excessive punctuation. We already removed `$XX` from gift card subjects.
6. **Test the actual delivery:** send a real test using a Gmail account you control, then "Show original" on the message and check the SPF / DKIM / DMARC results in the header.

Tools:

- mail-tester.com — send a test email, get a spam score
- mxtoolbox.com — DNS record sanity check

Customer didn't receive confirmation email

Symptoms: Order completed (`status: 'paid'` in Sanity). Customer says no email arrived. Not in spam either.

Diagnosis:

1. Resend Dashboard → Logs → search by recipient email
2. If the send is logged with status `delivered`: it's a deliverability issue, see spam section
3. If logged with status `bounced` or `failed`: the recipient's mail server rejected it. Common: invalid email typo, full mailbox, blocked sender.
4. If not logged at all: the webhook didn't reach the `resend.emails.send` call. Check the function log for `Order paid but no customer email to send confirmation` (means the Sanity order had no email field) or any Resend client error.

Fix:

- Bounced/typo: contact the customer through another channel, re-send manually from Studio after correcting the email
- Webhook didn't reach the send: see stuck pending

Sanity write fails (403 / token error)

Symptoms: Vercel logs: `Insufficient permissions; permission \"create\" required` or `403 Forbidden` from Sanity API.

Cause: `SANITY_API_TOKEN` exists but has Viewer / Developer permission, not Editor.

Fix:

1. Sanity Manage → Project → API → Tokens
2. Create a new token with **Editor** permission (or higher)
3. Update `SANITY_API_TOKEN` in Vercel for all scopes that need writes (all of them)
4. Redeploy

The old token can be deleted after the new one is confirmed working.

Cal.com booking embed is blank

Symptoms: `/book/[slug]` renders the page chrome but the booking widget is just empty.

Diagnosis:

1. Browser DevTools → Console → look for Cal.com errors
2. Network tab → check the Cal.com iframe URL — does it return 200?
3. Check `NEXT_PUBLIC_CAL_USERNAME` in the deployed env — typo here breaks everything
4. Verify the Cal.com event with that slug actually exists at `cal.com/stoneiwc/<slug>`

Common causes:

- Slug doesn't exist on Cal.com (event was deleted or renamed)
- `CAL_API_KEY` rotated and Vercel env wasn't updated
- The Cal.com account itself has billing/access issues

Fix:

- Re-provision the event with the Python script in `scripts/` (see cal-events.md)
- Or recreate it manually in Cal.com's UI with the same slug

ISR not updating after Sanity edit

Symptoms: You change a product price/name in Studio. The site still shows the old value > 60 seconds later.

Diagnosis:

- The page in question — does it `export const revalidate = 60`? Check the file.
- Is it actually being requested? ISR only refreshes on the first request after the revalidation period.

Fix:

- **Manual revalidation:** `POST /api/revalidate` with the appropriate tag. Or simpler: append `?cache_bust=1` to the URL to bypass once (just to confirm the data is correct in Sanity).
- **Sanity webhook integration:** `/api/revalidate` can be wired to a Sanity webhook so any document change triggers tag invalidation. Not configured by default.
- **Last resort:** Vercel → Deployments → "Redeploy" the production deploy without cache. This rebuilds everything fresh.

`vercel env pull` shows blank values

Symptoms: `vercel env pull .env.vercel` succeeds but the file has variables with empty values.

Cause: The Vercel dashboard hides sensitive variable values after save — they're encrypted at rest. `vercel env pull` does decrypt and write them locally, so a truly blank value means the variable was actually saved as empty.

Diagnosis:

1. Vercel → Settings → Environment Variables → click the … menu → "Edit" → the value field should be populated (Vercel UI sometimes masks it; click the eye icon to reveal)
2. If truly blank: someone saved it without typing a value. Re-enter.
3. Verify scope: if you `vercel env pull --environment=production` but the variable is only set for Preview, it'll come back blank.

See also: environments-and-deployments.md.

Production accidentally using test Stripe keys

Symptoms: Real customer "paid", but no charge shows up in Stripe Live mode Dashboard. Order is `paid` in Sanity but the money never moved.

This is bad — the customer thinks they paid; you can't fulfill the order without payment. Catch it before merge by following the production merge checklist.

Recovery:

1. Contact affected customers immediately
2. Provide a manual payment link (Stripe → Create payment link) for them to retry
3. Mark the original Sanity order as `cancelled` + `internalNotes` explaining

Prevention: Always verify production keys before announcing:

```
vercel env pull .env.production --environment=production
grep STRIPE_SECRET_KEY .env.production   # must start with sk_live_
```

Then delete `.env.production` (it's gitignored but treat it as a secret).

Site is down / 500 on every page

Symptoms: Production homepage returns 500. Was working before.

Diagnosis:

1. Vercel → Deployments → latest production deploy → "Logs" or "Building" tab
2. If the build failed: revert to the previous deploy via Vercel's "Promote to Production" on the last good build
3. If the build succeeded but runtime is failing: function logs will show the actual exception

Common causes:

- Sanity dataset became inaccessible (Sanity outage, project paused, token revoked)
- A required env var was deleted or replaced with a wrong value
- A recent code change throws on render

Fix:

- Roll back first (Vercel → previous deploy → "Promote to Production")
- Then investigate without the customer impact
- See environments-and-deployments.md for rollback caveats

Test scenarios

A reusable end-to-end test set for verifying the system after major changes. Run on `dev.stoneiwc.com` in Stripe Test mode.

Cards (Stripe test mode):

- Success: `4242 4242 4242 4242`
- Decline (insufficient funds): `4000 0000 0000 9995`
- Decline (3DS required, then succeed): `4000 0027 6000 3184`
- Any future expiry, any CVC, any ZIP

Scenarios:

#	Scenario	Expected
1	Normal order (no coupon, no gift card)	Sanity order `paid`, email received, Stripe metadata correct
2	Order with coupon	Coupon discount appears in cart UI, Sanity total reflects it, Stripe charge matches
3	Order with gift card	Gift card balance decreases by the applied amount, Sanity Order has `giftCardApplied`
4	Order with coupon + gift card	Both discounts apply, charge = `(subtotal − couponDiscount) + shipping − giftCardDiscount`
5	Gift card purchase	Recipient email arrives, purchaser confirmation email arrives, Sanity Gift Card created with status `active`
6	Failed payment (4000…9995)	Sanity Order flips to `failed`, gift card balance NOT decremented
7	Webhook retry / "Resend" in Stripe Dashboard	Order doesn't duplicate, gift card balance doesn't double-decrement, email doesn't re-send
8	Tampered gift card code	"Invalid code" error, checkout doesn't proceed
9	Tampered coupon code	"Invalid coupon" error
10	Cart sheet → View Cart → Checkout navigation	Coupon survives navigation, totals consistent

---

What to do when you can't figure it out

1. Capture: the time of failure, the PaymentIntent ID (if applicable), the order number (if applicable), the customer email
2. Grab the Vercel function logs around that time (Vercel → Logs → filter)
3. Grab the Stripe event delivery details (if applicable)
4. Run the relevant GROQ in Sanity Vision to confirm the document state

Then ask in the team Slack/email — the people who built this know the weird corners.

Sanity CMS — Technical Documentation

Overview

Sanity is the content management system for all dynamic content — products, categories, coupons, shipping methods, team members, articles, press items, and various page images. The Studio runs separately at `http://localhost:3333` (dev) and is deployed at its own URL in production.

All data access goes through GROQ queries in `lib/sanity.queries.ts`. The frontend never talks to the Sanity HTTP API directly — everything goes through `lib/sanity.client.ts`.

Key Files

File	Purpose
`lib/sanity.client.ts`	Sanity client initialization
`lib/sanity.queries.ts`	All GROQ queries and TypeScript types
`lib/sanity.image.ts`	Image URL builder (`urlFor`)
`studio/schemaTypes/`	Schema definitions for all content types
`studio/schemaTypes/index.ts`	Registers all schemas with the Studio

Client Setup (`lib/sanity.client.ts`)

Initialized with:

- `projectId` : `NEXT_PUBLIC_SANITY_PROJECT_ID`
- `dataset` : `NEXT_PUBLIC_SANITY_DATASET`
- `apiVersion` : `NEXT_PUBLIC_SANITY_API_VERSION`
- `useCdn: true` for reads (cached CDN)
- `token` : `SANITY_API_TOKEN` for write access (used in revalidation)

Caching & ISR

All Sanity queries use Next.js fetch cache tags for selective invalidation:

```
client.fetch(query, params, { next: { tags: ["product"] } })
```

Pages that use ISR:

Page	Revalidate
`/` (homepage)	60s
`/about`	60s
`/education`	60s
`/featured`	60s
`/products`	60s
`/products/[slug]`	60s
`/education/articles/[slug]`	60s

Cache invalidation: `POST /api/revalidate` can trigger on-demand revalidation using Sanity webhooks or manual calls.

Content Schemas

Product (`studio/schemaTypes/product.ts`)

The core e-commerce document type.

Field	Type	Required	Notes
`name`	string	✓	Display name
`slug`	slug	✓	Auto-generated from name, used in URL `/products/[slug]`
`price`	number	✓	Must be positive
`originalPrice`	number	—	Set for sale items; triggers strikethrough display
`shortDescription`	string	✓	Max 200 chars, shown on product cards
`description`	text	✓	Full text for product detail page
`image`	image	✓	With hotspot enabled + `alt` text field
`category`	reference → Category	✓	Linked category document
`tags`	string[]	—	e.g. `"bestseller"` , `"organic"` , `"new"`
`inStock`	boolean	—	Default: `true`
`featured`	boolean	—	Default: `false` ; shows on homepage featured section

Special product — Gift Certificate:

- Slug must be exactly `stoneiwc-gift-certificate`
- This slug is hardcoded in `components/products/gift-card-purchase.tsx` as `GIFT_CARD_SLUG`
- When this slug is detected, the product detail page renders a gift card purchase form instead of the standard add-to-cart UI
- The `price` field is ignored for this product — pricing is handled dynamically in the purchase form

Adding a new product:

1. Open Sanity Studio → Products → New
2. Fill all required fields
3. Assign a category (must exist first)
4. Set `featured: true` to appear on the homepage

5. The slug auto-generates from the name — change it if needed before publishing
6. The product page at `/products/[slug]` will be available after the next revalidation (max 60s)

Sale pricing: Set `originalPrice` to the original price and `price` to the sale price. The product card automatically shows:

- A "Sale" badge
- The sale price in full size
- The original price crossed out

Category (`studio/schemaTypes/category.ts`)

Referenced by products. Displayed as filter buttons on `/products`.

Field	Type	Notes
`name`	string	Display name, must be unique
`slug`	slug	URL-safe identifier
`description`	text	Optional
`orderRank`	number	Controls display order in filters

Categories are fetched with `getAllCategories()` which orders by `orderRank asc`. On the products page, "All" is prepended in the frontend — it is not a Sanity document.

Coupon (`studio/schemaTypes/coupon.ts`)

Discount codes applied during checkout. Validated server-side via `POST /api/validate-coupon`.

Field	Type	Notes
`code`	string	Case-sensitive code customers enter
`description`	string	Internal description
`type`	`"percentage"` | `"fixed"`	Percentage off or fixed dollar amount
`value`	number	% value or dollar amount
`minSubtotal`	number	Optional minimum cart subtotal
`isActive`	boolean	Default: `true`; set to `false` to disable

Only coupons with `isActive == true` are returned by `getCoupons()`.

Discount calculation:

- `percentage`: `subtotal × (value / 100)`
- `fixed`: `value` dollars off (capped at subtotal)

Shipping Method (`studio/schemaTypes/shippingMethod.ts`)

Available shipping options shown at checkout. Fetched via `GET /api/shipping-methods`.

Field	Type	Notes
`id`	slug	Used as the identifier in PaymentIntent metadata
`name`	string	Display name (e.g. "Standard Shipping")
`description`	string	Shown under the name at checkout
`cost`	number	Dollar amount ($0 = free)
`isActive`	boolean	Only active methods are shown
`order`	number	Controls display order

Other Content Types

These are simpler content types managed in Sanity. Most are single-document types or ordered lists.

Schema	Used On	Notes
`heroSlide`	Homepage hero carousel	`subtitle`, `title`, `description`, `image`, ordered by `orderRank`
`article`	`/education/articles`	`title`, `slug`, `publishedAt`, `coverImage`, `excerpt`, `tags`, `body` (rich text)
`teamMember`	About → Team	`name`, `title`, `bio`, `image`, `specialties`, ordered by `orderRank`
`partner`	About → Partners	`name`, `description`, `logo`, `websiteUrl`, ordered by `orderRank`
`pressItem`	Featured → Press	`title`, `description`, `image`, `link`, ordered by `orderRank`
`awardItem`	Featured → Awards	`title`, `description`, `image`, ordered by `orderRank`
`mediaItem`	Featured → Media	`title`, `description`, `youtubeUrl`, ordered by `orderRank`
`qcShowFlyer`	Featured → QC Show	Single document, image only
`qcShowEpisode`	Featured → QC Show	`youtubeUrl`, ordered by `_createdAt`

Singleton image documents (one document per page section):

Schema ID	Used On	Studio Path
`homePageImages`	Homepage about + culinary images	Pages → Home → Images
`aboutPageImages`	`/about` hero image	Pages → About Us → About Us Page → Images
`ourStoryImages`	About → Our Story	Pages → About Us → Our Story → Images
`servicesPageImages`	`/services` section images	Pages → Services → Services Page → Images
`conciergeImages`	Services → Concierge	Pages → Services → Concierge → Images
`virtualConsultationsImages`	Services → Virtual Consultations	Pages → Services → Virtual Consultations → Images
`educationPageImages`	`/education` hero image	Pages → Education → Education Page → Images
`certificationImages`	Education → Certifications	Pages → Education → Practitioner Certifications → Images
`licenseeProgramImages`	Education → Licensee Program	Pages → Education → Licensee Programs → Images
`cuppingImages`	Education → Cupping	Pages → Education → Cupping → Images
`featuredPageImages`	`/featured` hero image	Pages → Featured On → Featured On Page → Images
`qcShowFlyer`	Featured → QC Show	Pages → Featured On → QC Show → Flyer

These use a fixed `_id` (same as `_type`) so there is always exactly one document per type. Queried with `[0]` selector and the `_id` filter.

GROQ Queries (`lib/sanity.queries.ts`)

Product Projection

All product queries share a reusable projection:

```
_id,
name,
slug,
price,
originalPrice,
description,
shortDescription,
image,
"category": category->{name},
tags,
inStock,
featured
```

The `"category": category->{name}` syntax dereferences the category reference and returns only the `name` field. This is why the frontend `Product` type has `category: string` (the name), not an object.

Available Query Functions

Function	GROQ Filter	Cache Tag
`getAllProducts()`	`_type == "product"` ordered by `name`	`product`
`getProductBySlug(slug)`	`slug.current == $slug`	`product`
`getFeaturedProducts()`	`featured == true`	`product`
`getProductsByCategory(name)`	`category->name == $categoryName`	`product`
`getAllProductSlugs()`	Returns `slug.current[]` only	`product`
`getAllCategories()`	`_type == "category"` ordered by `orderRank`	`category`
`getCoupons()`	`isActive == true`	`coupon`
`getShippingMethods()`	`isActive == true` ordered by `order`	`shippingMethod`
`getArticles()`	ordered by `publishedAt desc`	`article`
`getArticleBySlug(slug)`	`slug.current == $slug`	`article`
`getAboutPageImages()`	singleton `aboutPageImages`	`aboutPageImages`
`getEducationPageImages()`	singleton `educationPageImages`	`educationPageImages`
`getFeaturedPageImages()`	singleton `featuredPageImages`	`featuredPageImages`

`transformProduct` Function

Sanity returns products as `SanityProduct` (raw Sanity shape). `transformProduct()` converts them to the frontend `Product` type used everywhere:

```
// Key transformations:
id: sanityProduct._id                          // Sanity _id → frontend id
image: urlFor(sanityProduct.image).width(800).height(800).url()  // image object → URL string
category: sanityProduct.category.name          // { name } object → string
tags: sanityProduct.tags || []                 // null-safe
```

Image Handling (`lib/sanity.image.ts`)

```
import { urlFor } from '@/lib/sanity.image'

urlFor(image).width(800).height(800).url()  // product images
urlFor(image).width(400).url()              // thumbnails
```

Sanity stores images as `{ asset: { _ref, _type } }` objects. `urlFor()` builds an optimized CDN URL using `@sanity/image-url`. Always specify dimensions to avoid serving oversized images.

Images are passed to Next.js `<Image>` which handles further optimization (WebP conversion, lazy loading).

Studio Structure (`studio/sanity.config.ts`)

The Studio sidebar is manually structured — new schema types do not appear automatically. The hierarchy mirrors the site navigation:

```
Pages
├── Home
│   ├── Hero Slides
│   └── Images (homePageImages)
├── About Us
│   ├── About Us Page → Images (aboutPageImages)
│   ├── Our Story → Images (ourStoryImages)
│   ├── Team Members
│   └── Partners & Affiliates
├── Services
│   ├── Services Page → Images (servicesPageImages)
│   ├── Concierge → Images (conciergeImages)
│   └── Virtual Consultations → Images (virtualConsultationsImages)
├── Education
│   ├── Education Page → Images (educationPageImages)
│   ├── Practitioner Certifications → Images (certificationImages)
│   ├── Licensee Programs → Images (licenseeProgramImages)
│   ├── Cupping → Images (cuppingImages)
│   └── Articles
└── Featured On
    ├── Featured On Page → Images (featuredPageImages)
    ├── Press
    ├── Media
    ├── Awards
    └── QC Show (Flyer + Episodes)

Products
└── Categories, All Products

Shipping & Coupons
└── Coupons, Shipping Methods
```

When adding a new schema type, it must be registered in both `studio/schemaTypes/index.ts` AND manually placed in the structure in `sanity.config.ts`. It must also be added to the exclusion filter at the bottom of `sanity.config.ts` to prevent it from appearing twice. If the document is a singleton, add it to the `singletonTypes` array in the same file.

Adding a New Schema Type

1. Create `studio/schemaTypes/{typeName}.ts`
2. Define fields with `defineType` + `defineField`
3. Export the type and import it in `studio/schemaTypes/index.ts`
4. Add to the `schemaTypes` array in `index.ts`
5. Add a `S.listItem()` entry in the correct section of `sanity.config.ts`
6. Add the type name to the exclusion filter array at the bottom of `sanity.config.ts`
7. If singleton: add to `singletonTypes` in `sanity.config.ts` and use a fixed `documentId` matching `_type`
8. Write a query function in `lib/sanity.queries.ts` with an appropriate cache tag
9. Deploy the Studio: `cd studio && npx sanity deploy`

> Schema changes are non-destructive — adding fields does not affect existing documents. Removing or renaming fields will cause existing data to silently disappear from queries.

Checkout & Payments — Technical Documentation

Overview

The checkout system combines a client-side cart (React Context + localStorage), Sanity-managed coupons and shipping methods, Stripe for payment processing, and a webhook handler that triggers post-payment side effects (gift card code delivery).

Cart System

File: `lib/cart-context.tsx`

The cart is a React Context backed by a `useReducer`. It persists to `localStorage` under the key `stone-iwc-cart` so the cart survives page refreshes.

State shape:

```
{
  items: CartItem[]          // { product: Product, quantity: number }[]
  isOpen: boolean            // drawer open/closed
  appliedCoupon: AppliedCoupon | null
}
```

Derived values (computed in context):

```
totalItems     // sum of all quantities
subtotal       // sum of (price × quantity) for all items
discountAmount // coupon discount applied to subtotal
totalPrice     // subtotal − discountAmount
```

Available actions:

- `addItem(product, quantity?)` — adds or increments
- `removeItem(productId)`
- `updateQuantity(productId, quantity)` — quantity 0 removes the item
- `clearCart()` — called on checkout success
- `applyCoupon(coupon)` / `removeCoupon()`
- `openCart()` / `closeCart()`

> The cart does **not** store gift card state — gift cards are applied at the checkout page level, not in the cart context.

Coupon System

Coupons are managed in Sanity (type: `coupon`) and validated server-side.

Sanity fields: `code`, `description`, `type` (`percentage` | `fixed`), `value`, `minSubtotal`, `isActive`

Validation endpoint: `POST /api/validate-coupon`

```
// Request
{ "code": "SUMMER20", "subtotal": 85.00 }

// Success response
{ "code": "SUMMER20", "description": "...", "type": "percentage", "value": 20 }

// Error responses
{ "error": "Coupon not found" }
{ "error": "Minimum subtotal of $50.00 required" }
```

Discount calculation (in cart context):

- `percentage`: `subtotal × (value / 100)`
- `fixed`: `value` (capped at subtotal)

Checkout Flow — End to End

Step 1: Checkout Page (`app/checkout/page.tsx`)

Server-side: empty. All state lives client-side.

Local state managed here:

- `shippingMethod` + `shippingCost` (updated by `SelfCheckoutSection`)
- `appliedGiftCard: { code, amount, promotionCodeId } | null`

Final total formula:

```
giftCardDiscount = Math.min(appliedGiftCard.amount, subtotal − couponDiscount + shippingCost)
finalTotal = Math.max(0, subtotal − couponDiscount + shippingCost − giftCardDiscount)
```

Both `SelfCheckoutSection` (left column) and `CheckoutDetailsSection` (right column) receive `finalTotal`, `appliedGiftCard`, and `giftCardDiscount` as props.

Step 2: Customer Form (`SelfCheckoutSection.tsx`)

Collects:

- Email, first name, last name, phone
- Billing address (line1, line2, city, state, postal code, country)
- Shipping address (or "same as billing" toggle)
- Shipping method (radio list fetched from `/api/shipping-methods`)
- Gift card code input (validated live against `/api/gift-cards/validate`)

On "Continue to Payment":

1. `POST /api/checkout/retrieve-stripe-publishable-key` → gets `publishableKey`
2. `POST /api/checkout/create-stripe-payment-intent` → gets `clientSecret`
3. Stripe Elements (`<PaymentElement>`) renders

Step 3: Payment Intent Creation (`app/api/checkout/create-stripe-payment-intent/route.tsx`)

Request body:

```
{
  items: { id, name, price, quantity }[]
  shippingMethod: string
  shippingCost: number
  taxAmount?: number
  processingFee?: number
  shippingAddress: { firstName, lastName, address, city, state, zipCode, country }
  billingAddress: { firstName, lastName, address, city, state, zipCode, country }
  totalAmount: number          // already discounted final total
  email: string
  giftCardCode?: string        // if a gift card was applied
  giftCardPromotionCodeId?: string
}
```

What gets created in Stripe:

```
PaymentIntent {
  amount: totalAmount × 100,   // in cents
  currency: "usd",
  receipt_email: email,
  metadata: {
    order_type: "storefront",
    items_count: "3",
    customer_email: "...",
    shipping_method: "standard",
    shipping_cost: "9.99",
    processing_fee: "0.00",
    tax_amount: "0.00",
    gift_card_code: "STONE-XXXX-XXXX",          // if applied
    gift_card_promotion_code_id: "promo_xxx"     // if applied
  }
}
```

> `totalAmount` already has the gift card discount subtracted — the PaymentIntent amount is the actual charge to the customer's card.

Step 4: Payment Confirmation

Stripe Elements handles card input. On submit:

```
await elements.submit()
await stripe.confirmPayment({ elements, redirect: 'if_required', ... })
```

On success:

- Redirects to `/checkout/success?payment_intent=pi_xxx`
- Success page calls `GET /api/checkout/retrieve-stripe-payment-intent-status?payment_intent=pi_xxx`
- Verifies `status === "succeeded"`, shows confirmation, clears cart

Step 5: Stripe Webhook (`app/api/webhooks/stripe/route.ts`)

Fires on `payment_intent.succeeded`. Two responsibilities:

A) Gift card redemption (any order type):

- If `metadata.gift_card_promotion_code_id` exists → `stripe.promotionCodes.update(id, { active: false })`
- This prevents the code from being reused

B) Gift card creation (gift card orders only):

- If `metadata.order_type === "gift_card"` → generate `STONE-XXXX-XXXX` code, create Stripe Coupon + PromotionCode, send email via Resend

See `project-documentation/gift-card.md` for full webhook details.

Order Summary (`CheckoutDetailsSection.tsx`)

Displays in order:

1. Item list (name, qty, price)
2. Subtotal
3. Coupon discount (if any) — `Coupon (CODE): -$XX`
4. Gift card discount (if any) — `Gift Card (STONE-XXXX): -$XX`
5. Shipping (method name + cost)
6. **Total**

Shipping Methods

Fetched from Sanity at checkout load time via `GET /api/shipping-methods`.

Sanity fields: `id` (slug), `name`, `description`, `cost`, `isActive`, `order`

Only `isActive == true` methods are returned, sorted by the `order` field.

The selected method ID is saved to `localStorage` (`selectedShippingMethod`) so it persists across the checkout steps. Cleared on successful payment.

Local Development

Stripe webhook cannot be tested on `localhost` without forwarding. Options:

1. **Stripe CLI** (recommended for local):

   ```
   stripe listen --forward-to localhost:3000/api/webhooks/stripe
   ```

   Copy the printed `whsec_xxx` into `.env.local` as `STRIPE_WEBHOOK_SECRET`.

2. **Deployed preview URL** (`dev.stoneiwc.com`): Register the webhook endpoint in Stripe Dashboard → Developers → Webhooks. See the gift card deploy checklist for full steps.

Test card: `4242 4242 4242 4242`, any future expiry, any CVC.

Order System — Technical Documentation

Overview

Storefront orders are persisted in **Sanity** as `order` documents so admins can manage fulfillment from Sanity Studio without ever opening the Stripe Dashboard. Stripe remains the payment processor and the canonical source for the *charge*, but everything else — addresses, items, status, tracking, internal notes — lives in Sanity.

Gift card purchases (slug `stoneiwc-gift-certificate`) follow a separate flow and do **not** create order documents — see gift-card.md.

Flow Diagram

```
[Customer fills checkout form]
        │
        ▼
[POST /api/checkout/create-stripe-payment-intent]
        ├── Creates a Stripe PaymentIntent (rich metadata: address, items_summary, gift card)
        └── Creates a Sanity `order` document, status='pending', linked by stripePaymentIntentId
        │
        ▼
[Stripe Elements renders payment form]
        │
        ▼
[Customer enters card → Stripe processes payment]
        │
        ├── Success → webhook fires `payment_intent.succeeded`
        │       ├── markOrderPaid(paymentIntentId)  → status='paid', paidAt=now
        │       └── If gift card applied: redeemGiftCard(code, amount, paymentIntentId)
        │
        └── Failure → webhook fires `payment_intent.payment_failed`
                └── markOrderFailed(paymentIntentId) → status='failed'
```

Gift cards never cover shipping (capped at `subtotal − couponDiscount`), so every order produces a real Stripe charge — there is no zero-total fallback.

File Structure

```
app/api/
├── checkout/
│   └── create-stripe-payment-intent/route.tsx   ← Creates PI + Sanity order
└── webhooks/
    └── stripe/route.ts                          ← Marks paid/failed, redeems gift cards

lib/
├── orders.ts                                    ← Sanity helpers (create/markPaid/markFailed)
└── sanity.client.ts                             ← Shared write-enabled client

studio/schemaTypes/
└── order.ts                                     ← Sanity document type
```

Sanity `order` Document

Visible in Sanity Studio under **Order**. Fields grouped into:

Overview

Field	Type	Editable	Notes
`orderNumber`	string	read-only	`SIWC-YYYYMMDD-XXXX`
`status`	enum	yes	pending / paid / shipped / delivered / cancelled / refunded / failed
`createdAt`	datetime	read-only	When PaymentIntent was created
`paidAt`	datetime	read-only	Set by webhook on success
`stripePaymentIntentId`	string	read-only	Link back to Stripe

Fulfillment (admin-editable)

Field	Type	Notes
`shippedAt`	datetime	Admin sets when package leaves
`trackingCarrier`	string	UPS, USPS, FedEx, etc.
`trackingNumber`	string	Provided by carrier

Customer

Field	Notes
`customer`	{ email, firstName, lastName, phone? }
`shippingAddress`	{ line1, line2, city, state, postalCode, country }
`billingAddress`	Same shape
`billingSameAsShipping`	boolean denormalized for quick UI

Items & totals

Field	Notes
`items[]`	{ productId, name, slug, quantity, unitPrice, lineTotal, image }
`subtotal`	Pre-discount merchandise total
`couponCode`, `couponDiscount`	If a coupon was applied
`shippingMethod`, `shippingCost`	
`giftCardCode`, `giftCardApplied`	If a gift card covered part of the order
`total`	What was actually charged to the card

Internal

Field	Notes
`internalNotes`	Free-form text for admin notes (fulfillment, customer service)

The Studio preview shows `SIWC-... 💳 $123.50 · customer@example.com`. Filter and sort by status, date, or total.

File-by-File

`lib/orders.ts`

Server-only helpers wrapping the Sanity client:

- `generateOrderNumber(now?)` — `SIWC-YYYYMMDD-XXXX`. The random suffix uses the same 32-char unambiguous alphabet as gift cards.
- `createPendingOrder(input)` — idempotent by `stripePaymentIntentId` (returns the existing doc if it's already there). Retries up to 5 times on `orderNumber` collision.
- `markOrderPaid(paymentIntentId)` — flips `pending` (or `failed`, in case of a retry after a transient failure) to `paid` and stamps `paidAt`. Never downgrades from `shipped` / `delivered`.
- `markOrderFailed(paymentIntentId)` — flips only `pending` to `failed`. Does not overwrite finalized states.

`app/api/checkout/create-stripe-payment-intent/route.tsx`

1. Creates the Stripe PaymentIntent with rich metadata (`ship_line1`, `ship_city`, `items_summary`, optional `coupon_code`, `gift_card_code`, …) so admins can read order details in the Stripe Dashboard too.
2. Calls `createPendingOrder()` to mirror the order into Sanity.
3. The Sanity write is best-effort: if it fails, the customer can still pay (`clientSecret` is returned). Failures are logged with the PaymentIntent ID so admins can recover from Stripe Dashboard data.

`app/api/webhooks/stripe/route.ts`

Listens for two events:

- `payment_intent.succeeded` → if `metadata.order_type === 'storefront'`, calls `markOrderPaid()`. Then runs the existing gift card redemption and gift card purchase logic.
- `payment_intent.payment_failed` → calls `markOrderFailed()` for storefront orders.

Both Sanity writes are wrapped in try/catch — webhook always returns 200 to Stripe so it doesn't keep retrying for non-critical failures.

`components/checkout/SelfCheckoutSection.tsx`

The checkout form now forwards everything the order document needs:

- `subtotal`, `couponCode`, `couponDiscount`
- `firstName`, `lastName`, `phone` (top-level customer fields, in addition to the address blocks)
- Each cart item: `id`, `name`, `slug`, `image`, `price`, `quantity`
- `billingSameAsShipping` flag

Admin Workflow

Day-to-day fulfillment in Sanity Studio:

1. Open **Order** in the sidebar. Sort by "Newest first" or "Status, then newest".
2. Click an order to see the full breakdown — address, items, payment split.
3. To ship:
   - Set `shippedAt` (date+time)
   - Fill `trackingCarrier` (e.g. `USPS`) and `trackingNumber`
   - Change `status` to `shipped`
4. When the package arrives (if you track this), set `status` to `delivered`.
5. For refunds: issue the refund in Stripe Dashboard, then set `status` to `refunded` in Sanity. (Sanity is the customer-facing record of truth.)
6. `internalNotes` is free-form — useful for "customer called, leave at back door" type notes.

> Tip: Tracking links can be opened by copying the number into the carrier's website. We don't auto-link yet.

Failure Modes & Recovery

Sanity write fails during checkout — customer still pays. The PaymentIntent has no matching Sanity order. Recovery: use the Stripe Dashboard's PaymentIntent detail page to read the rich metadata (`ship_*`, `items_summary`, `customer_email`) and recreate the order in Sanity manually. The webhook will log `markOrderPaid: no order for PaymentIntent ...` when it can't find a document.

Webhook hasn't fired yet — order stays at `status: 'pending'` past the success page. Stripe usually delivers within seconds; if it's stuck, replay the event from Stripe Dashboard → Webhooks → Event deliveries → "Resend".

Abandoned cart — customer initiated payment but never confirmed. The pending order sits in Sanity. There's no auto-cleanup. Either filter pending+age in the Studio query and bulk-delete, or set up a Vercel Cron job that deletes pending orders older than 24h. (Not implemented yet.)

Concurrent writes on the same PaymentIntent — `createPendingOrder` checks for existing by `stripePaymentIntentId` and returns the existing doc instead of creating a duplicate. Stripe retrying its webhook delivery is safe.

Environment Variables

Order tracking adds no new env vars — uses the existing Sanity setup. The `SANITY_API_TOKEN` must have **Editor** permission (the same token already required for gift card creation).

Future Work

- **Abandoned cart cleanup cron** — Vercel Cron that deletes `pending` orders older than 24 hours.
- **Order confirmation email** — Currently the customer only gets Stripe's automatic receipt. A custom Resend email with order details and tracking number could be sent on `status='paid'` and `status='shipped'`.
- **Customer order history page** — `/account/orders` would query Sanity by `customer.email` (after auth is added).
- **Server-side price validation** — Today the server trusts client-sent item prices. A future hardening pass should re-fetch prices from Sanity at `createPendingOrder` time and reject mismatches.

Gift Card System — Technical Documentation

Overview

Gift cards at Stone IWC use **Sanity as the source of truth** for state (code, balance, status, redemption history) and **Stripe** for payment processing. Each gift card supports **partial redemption** — buying a $100 card and using $30 leaves a $70 balance under the same code.

Gift cards apply to **merchandise only** (`subtotal − couponDiscount`). Shipping is always paid by the customer, which guarantees every order produces a Stripe charge above the $0.50 minimum. Storefront orders are mirrored into Sanity for fulfillment — see orders.md.

A purchase collects sender name (required), purchaser email, optional recipient name, recipient email, and an optional 500-char personal note. The recipient receives a styled email with the code and the message; if the recipient differs from the purchaser, the purchaser also receives a confirmation email (no code).

Flow Diagram

```
[Customer fills the purchase form]
        │ amount + sender name + emails + (optional note/recipient name)
        ▼
[create-payment-intent API] ── Creates a Stripe PaymentIntent
        │                       metadata: order_type=gift_card
        │                                 sender_name, recipient_name,
        │                                 customer_email, recipient_email,
        │                                 gift_card_note, gift_card_amount
        ▼
[Stripe processes the payment]
        ▼
[Webhook fires: payment_intent.succeeded]
        ├── (Idempotent) Create Sanity giftCard document with code, balance,
        │       purchaser/recipient info, optional note, expiresAt (+1 year)
        ├── Resend → email to recipient: code + value + note
        └── If purchaser ≠ recipient → Resend → confirmation email to purchaser

[Recipient enters code at checkout]
        ▼
[validate API queries Sanity]
        ▼
[If active + has balance + not expired, returns code + balance]
        ▼
[Checkout shows applied discount = min(balance, orderTotal)]
        │ excess balance stays on the card
        ▼
[After payment succeeds, webhook deducts the applied amount from Sanity]
        │ if balance reaches 0, status flips to 'redeemed'
```

File Structure

```
app/
├── api/
│   ├── gift-cards/
```

```
│   │   ├── create-payment-intent/route.ts   ← Purchase PaymentIntent
│   │   └── validate/route.ts                ← Looks up code + balance in Sanity
│   └── webhooks/
│       └── stripe/route.ts                  ← Creates Sanity doc, deducts on redeem
│
components/
└── products/
    └── gift-card-purchase.tsx               ← Purchase form (amount, names, note)

lib/
├── gift-cards.ts                            ← Sanity helpers (create, get, redeem)
└── email/
    └── gift-card-template.ts                ← Recipient + purchaser email templates

studio/
└── schemaTypes/
    └── giftCard.ts                          ← Sanity document type
```

Sanity `giftCard` Document

Visible in Sanity Studio under "Gift Card". Fields:

Field	Type	Notes
`code`	string	`STONE-XXXX-XXXX` format, generated server-side
`originalAmount`	number	What the purchaser paid (USD)
`currentBalance`	number	Remaining unspent balance
`status`	`'active'` | `'redeemed'` | `'expired'`	Auto-updates when balance reaches 0
`purchaserName`	string	Required from purchase form
`purchaserEmail`	string	Required
`recipientName`	string?	Optional
`recipientEmail`	string	Required
`note`	text?	Optional, max 500 chars
`paymentIntentId`	string	Stripe PaymentIntent that funded the card
`expiresAt`	datetime	+1 year from purchase
`redemptions`	array	`{ orderId, amount, redeemedAt }` per use

Studio preview shows `STONE-XXXX-XXXX ✅ $100 → $40 left · recipient@example.com`.

File-by-File Breakdown

1. `app/api/gift-cards/create-payment-intent/route.ts`

Creates a Stripe PaymentIntent for the purchase. Required: `amount` ($1–$500), `senderName`, `purchaserEmail`, `recipientEmail`. Optional: `recipientName`, `note` (≤ 500 chars). Writes everything to PaymentIntent metadata so the webhook has it on success.

2. `app/api/webhooks/stripe/route.ts`

Listens for `payment_intent.succeeded`. Two responsibilities:

A) Gift card purchase — when `metadata.order_type === 'gift_card'`:

- Idempotency check: looks up any existing giftCard with this `paymentIntentId`. If found, reuses its `code`.
- Otherwise calls `createGiftCard()` (which generates a unique `STONE-XXXX-XXXX`).
- Sends the recipient email (code + note) and, if buyer ≠ recipient, a purchaser confirmation email.

B) Gift card redemption — when `metadata.gift_card_code` and `metadata.gift_card_applied_amount` are present (any order, not just gift card purchases):

- Calls `redeemGiftCard(code, appliedAmount, paymentIntentId)`.
- Atomically deducts from `currentBalance`, appends to `redemptions`, and sets `status='redeemed'` if balance reaches 0.

Signature verification uses `stripe.webhooks.constructEvent` with `STRIPE_WEBHOOK_SECRET`.

3. `app/api/gift-cards/validate/route.ts`

`GET /api/gift-cards/validate?code=STONE-XXXX-XXXX`. Queries Sanity via `getGiftCardByCode()`. Returns `{ valid, code, balance, originalAmount }` on success, or `{ valid: false, error }` if not found / no balance / expired.

4. `lib/gift-cards.ts`

Server-only helpers wrapping the Sanity client:

- `generateGiftCardCode()` — random `STONE-XXXX-XXXX` (excludes ambiguous `O/I/0/1`).
- `createGiftCard(input)` — generates a unique code (5 retries on collision) and writes the document.
- `getGiftCardByCode(code)` — single GROQ fetch, normalized to uppercase.
- `redeemGiftCard(code, amount, orderId)` — uses Sanity's `ifRevisionId` for optimistic concurrency. Concurrent redemptions of the same card will surface as a commit error (caller can retry).

5. `components/products/gift-card-purchase.tsx`

The purchase UI on `/products/stoneiwc-gift-certificate`. Two steps:

- **Step 1** — amount selector (preset $25/$50/$100/$150/$200 or custom up to $500), "From" block (sender name + purchaser email), "To" block (optional recipient name + recipient email), optional personal note with live character counter.
- **Step 2** — Stripe Elements payment form.

`GIFT_CARD_SLUG = 'stoneiwc-gift-certificate'` lives in `@/lib/constants` and is the single source of truth for the product slug.

6. `lib/email/gift-card-template.ts`

Two pairs of HTML/text templates:

- **Recipient email** (`giftCardEmailHtml` / `giftCardEmailText`) — greeting personalized with `recipientName` when present, `senderName` as the giver, optional note rendered in a gold-bordered block, the code, value and expiry. Mentions that unused balance remains.
- **Purchaser confirmation** (`giftCardPurchaseConfirmationHtml` / `giftCardPurchaseConfirmationText`) — sent only when purchaser ≠ recipient. Shows recipient name + email and the value. The code is **never** included for security.

All HTML interpolation goes through an `escapeHtml` helper.

Checkout Integration

`app/checkout/page.tsx`

- `AppliedGiftCard = { code, balance }` (balance is the remaining unspent amount returned by `validate`).
- `giftCardDiscount = min(balance, subtotal − couponDiscount + shippingCost)` — the card is never charged more than the order total.
- `finalTotal = subtotal − couponDiscount + shippingCost − giftCardDiscount`.

`components/checkout/SelfCheckoutSection.tsx`

- Apply: calls `/api/gift-cards/validate`, on success stores `{ code, balance }`.
- The applied-amount line shows `−$X applied` plus `· $Y balance remaining` when the card has unspent balance after this order.
- `initializePayment` sends `giftCardCode` and `giftCardAppliedAmount` (the actual discount, not the balance) to the checkout payment-intent API.

`app/api/checkout/create-stripe-payment-intent/route.tsx`

- Accepts `giftCardCode` and `giftCardAppliedAmount` from the body.
- Writes them to PaymentIntent metadata as `gift_card_code` and `gift_card_applied_amount`. The webhook reads these on success to deduct the balance.

`components/checkout/CheckoutDetailsSection.tsx`

- Shows `Gift Card (CODE)` and `−$X.XX` discount line in the order summary.

Environment Variables

```
# Sanity (needed for both read and write — token must allow writes)
NEXT_PUBLIC_SANITY_PROJECT_ID=
NEXT_PUBLIC_SANITY_DATASET=
NEXT_PUBLIC_SANITY_API_VERSION=
SANITY_API_TOKEN=                    # write-capable token

# Stripe
STRIPE_SECRET_KEY=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
STRIPE_WEBHOOK_SECRET=

# Resend
RESEND_API_KEY=
RESEND_FROM_EMAIL=

# App
NEXT_PUBLIC_FRONTEND_URL=
```

Production Deploy Checklist

1. Merge feature branch into `development` (auto-deploys to `dev.stoneiwc.com`).
2. Stripe Dashboard (test mode) → Webhooks → endpoint:

- URL: `https://dev.stoneiwc.com/api/webhooks/stripe?x-vercel-protection-bypass=<bypass-secret>`
- Listen for `payment_intent.succeeded`.
- Copy the signing secret into Vercel `STRIPE_WEBHOOK_SECRET` (Preview scope).

3. Verify `SANITY_API_TOKEN` (write-capable) is set on the Preview environment in Vercel.
4. Test with `4242 4242 4242 4242`: purchase → recipient email arrives → Sanity Studio shows the new gift card → apply at checkout → balance deducts on success.
5. Repeat the webhook setup for production (live mode → `stoneiwc.com`) before going live.

Troubleshooting

Webhook fired but no Sanity doc and no email — most often `SANITY_API_TOKEN` is missing on the Preview/Production environment, or the token lacks write permission. Check Vercel function logs for `Gift card Sanity creation error:`.

Code applies as $0 at checkout — happens if the validate API returns a stale or zero balance. Check the giftCard document in Sanity Studio: confirm `currentBalance > 0` and `status === 'active'`.

Recipient never got the email but Sanity doc exists — Resend domain not verified, the recipient address bounced, or the message was filtered as spam. Check Resend Dashboard → Emails for the delivery status. Verify SPF/DKIM/DMARC for `RESEND_FROM_EMAIL`.

Webhook returns 401 — `dev.stoneiwc.com` is behind Vercel's Deployment Protection. Use the `?x-vercel-protection-bypass=<secret>` query parameter on the Stripe webhook URL. The secret is configured under Project → Settings → Deployment Protection → Protection Bypass for Automation.

Concurrent redemptions race — `redeemGiftCard` uses Sanity's `ifRevisionId` for optimistic concurrency. If two webhook events deduct the same card simultaneously, one will throw on commit. Stripe will retry the failed webhook; the second attempt will see the updated revision and apply cleanly.

Local development: Stripe CLI — run `stripe listen --forward-to localhost:3000/api/webhooks/stripe` and put the printed `whsec_...` in `.env.local` as `STRIPE_WEBHOOK_SECRET`. Restart `pnpm dev` after the change.

Design Notes

- **Why Sanity, not a relational DB?** The project already runs Sanity for content. Adding a single document type avoids introducing a new dependency, gives non-engineers Studio access for support cases, and the redemption volume is low enough that Sanity's mutation API is more than adequate.
- **Why drop the Stripe Coupon/PromotionCode?** Stripe Coupons have a fixed `amount_off` that can't be changed after creation, which is incompatible with balance-based redemption. We now use Stripe purely for payment processing.
- **Why an `escapeHtml` helper in the email template?** The personal note is free-form user input that goes directly into an HTML email body. Without escaping, a malicious or accidental `<script>` would execute in some mail clients.

Cal.com Booking — Technical Documentation

Overview

The booking system uses the **Cal.com v2 REST API** directly — no SDK. Event types (services) live in Cal.com and are fetched at runtime with `cache: "no-store"`. The frontend filters and displays them; clicking a service embeds the Cal.com booking widget.

Service appointments are provisioned to Cal.com using a set of **Python scripts** in `scripts/`. These scripts are the source of truth for what event types exist — not the database, not Sanity.

Frontend Architecture

Pages

Page	File	Description
`/book`	`app/book/page.tsx`	Browse all services with category sidebar and search
`/book/[slug]`	`app/book/[slug]/page.tsx`	Individual service detail + Cal.com booking embed

Key Files

File	Purpose
`lib/cal-api.ts`	API client, category definitions, filter functions
`components/cal-booker.tsx`	Wraps `@calcom/embed-react` for the booking widget

API Client (`lib/cal-api.ts`)

Base URL: `https://api.cal.com/v2`
API Version header: `cal-api-version: 2024-06-14`
Cache policy: `cache: "no-store"` — always fetches fresh event types (availability changes in real time)

```
// Fetch all event types for the configured Cal.com username
await getEventTypes(process.env.NEXT_PUBLIC_CAL_USERNAME)
```

The response is `data.data[]` — an array of event type objects. Returns `[]` on any error (no throws).

`CalEventType` shape:

```
{
  id: number
  title: string
  slug: string
  description: string | null
  lengthInMinutes: number
  price: number      // in cents (e.g. 10000 = $100.00)
```

```
  currency: string
}
```

Booking Categories

Defined in `BOOKING_CATEGORIES` array in `lib/cal-api.ts`. Each category maps a `value` to a `slugPrefix`:

Category Value	Label	Slug Prefix
`hair`	Hair	`hair-`
`face`	Face	`face-`
`lash-extension`	Lash Extension	`lash-extension-`
`micropigmentation`	Micropigmentation Semi-Permanent	`micropigmentation-`
`skin-imperfection`	Skin Imperfection	`skin-imperfection-`
`acupuncture`	Acupuncture	`acupuncture-`
`cupping`	Cupping	`cupping-`
`massage-body`	Massage & Body Treatments	`massage-body-`
`chiropractic`	Chiropractic	`chiropractic-`
`body-transformation`	Body Transformation	`body-transformation-`
`lipo-treatments`	Lipo Treatments	`lipo-treatments-`
`waxing`	Waxing	`waxing-`
`chronic-venous-insufficiency`	Chronic Venous Insufficiency	`chronic-venous-insufficiency-`
`nail`	Nail	`nail-`
`wellness`	Wellness	`wellness-`
`consultation`	Consultation	`consultation-`
`concierge`	Concierge	`concierge-`

Category detection:

```
getCategoryFromSlug("hair-woman-haircut-short-style") // → "hair"
getCategoryFromSlug("face-tmj")                       // → "face"
```

Uses `slug.startsWith(cat.slugPrefix)` — first match wins.

Slug Conventions

Slugs are the single most important convention in this system. They drive category detection, pricing display, and badge rendering.

Standard services

```
{category-prefix}-{service-name}

Examples:
hair-woman-haircut-long-style
face-manual-microdermabrasion
nail-gel-manicure
acupuncture-full-body-session
```

Free services

```
{category-prefix}-{service-name}-free

Examples:
consultation-initial-free
hair-consultation-free
```

Detection: `event.slug.endsWith("-free")` OR `event.price === 0`
Display: shows **"Free"** badge instead of a price

Consultation-required services

```
{category-prefix}-consultation-{service-name}

Examples:
skin-imperfection-consultation-cherry-angioma
micropigmentation-consultation-scalp-micropigmentation
wellness-consultation-stoneiwc-cooking-on-site
```

Detection: `event.slug.includes("-consultation-")`
Display: shows amber **"Consultation Service"** badge and **"Price for Consultation"** label

> This pattern means the service requires a prior consultation before it can be booked. The event is created with `price: 0` and the description says "A consultation is required before booking this service."

Pricing fallback

If none of the above match and `price > 0` : display `$XX.XX`
If price is 0 but slug doesn't end in `-free` : display **"Contact for Pricing"**

Cal.com Provisioning Scripts

Overview

The Python scripts in `scripts/` create and update event types in Cal.com via the API. Each script corresponds to one service category. **Run these scripts when adding, updating, or restoring services in Cal.com** — do not create event types manually in the Cal.com dashboard unless you also update the script.

Setup

```
cd scripts
pip install -r requirements.txt
# requirements.txt contains: requests, python-dotenv
```

Scripts read `CAL_API_KEY` from the project root's `.env.local` automatically via `config.py`.

Shared Config (`config.py`)

```
BASE_URL = "https://api.cal.com/v2"
LOCATION_ADDRESS = "1108 W Parker Rd Ste 102 Plano, TX 75075"
WORK_HOURS_SCHEDULE_ID = 1493459          # Standard in-clinic schedule
VIRTUAL_CONSULTATION_SCHEDULE_ID = 1493460 # Virtual/remote schedule
HEADERS = { "Authorization": f"Bearer {API_KEY}", "cal-api-version": "2024-06-14" }
```

> `WORK_HOURS_SCHEDULE_ID` and `VIRTUAL_CONSULTATION_SCHEDULE_ID` are Cal.com internal IDs. If the Cal.com account is ever recreated, these IDs will change and must be updated in `config.py`.

Standard Event Payload

Every script sends the same base payload:

```
{
  "title": "...",
  "slug": "...",
  "lengthInMinutes": 60,
  "description": "...",
  "locations": [{ "type": "address", "address": LOCATION_ADDRESS, "public": True }],
  "price": 10000,               # in cents
  "currency": "usd",
  "scheduleId": WORK_HOURS_SCHEDULE_ID,
  "minimumBookingNotice": 2880, # 48 hours in minutes
  "beforeEventBuffer": 15,      # 15 min prep before
  "afterEventBuffer": 15,       # 15 min cleanup after
}
```

Running a Script

```
cd scripts
python create_hair_events.py
# Output:
# Creating 6 hair services...
# ✓ Woman Haircut Short & Style — $95.00 — https://cal.com/stoneiwc/hair-woman-haircut-short-style
# ✓ Men Haircut Short & Style — $65.00 — ...
```

Script Reference

Script	Category	Event Count	Notes
`create_hair_events.py`	Hair	6	Standard create
`create_face_events.py`	Face	7	Includes CPT codes in descriptions
`create_lash_extension_events.py`	Lash Extension	2	Standard create
`create_micropigmentation_events.py`	Micropigmentation	9	Standard create
`create_skin_imperfection_events.py`	Skin Imperfection	1	Standard create
`create_acupuncture_events.py`	Acupuncture	4	Standard create
`create_cupping_events.py`	Cupping	6	Standard create
`create_massage_body_events.py`	Massage & Body	2	Standard create
`create_chiropractic_events.py`	Chiropractic	1	Standard create
`create_body_transformation_events.py`	Body Transformation	6	Standard create
`create_lipo_treatments_events.py`	Lipo Treatments	10	Standard create
`create_waxing_events.py`	Waxing	7	Standard create
`create_cvi_events.py`	Chronic Venous Insufficiency	1	Standard create
`create_nail_events.py`	Nail	10	Standard create
`create_general_events.py`	Wellness / Consultation	4	Standard create
`create_consultation_required_events.py`	Mixed	7	**Upsert logic** (see below)
`update_hair_notice.py`	Hair	—	Update only, no new events

Upsert vs Create

Most scripts use **create-only** logic via `POST /v2/event-types`. Running them twice will create duplicate events.

`create_consultation_required_events.py` uses **upsert logic**:

1. Fetches all existing event types (`GET /v2/event-types` , paginated)
2. Builds a `slug → id` map
3. For each service: if slug exists → `PATCH /v2/event-types/{id}` , otherwise `POST /v2/event-types`

This script is safe to re-run at any time without creating duplicates. Use it as a reference pattern if you need to write a new updatable script.

Adding a New Service

1. Choose the correct category prefix from the `BOOKING_CATEGORIES` table above
2. Pick a slug: `{prefix}-{kebab-case-name}` (add `-free` if free, `-consultation-` if consultation-required)
3. Add the service dict to the appropriate script's list (or create a new script)
4. Run the script: `python create_{category}_events.py`
5. Verify the event appears at `https://cal.com/{username}/{slug}`

> **Prices are in cents.** `$95.00 → 9500` , `$150.00 → 15000` , free → `0` .

Adding a New Category

If a new service category is added to Cal.com:

1. Define a new entry in `BOOKING_CATEGORIES` in `lib/cal-api.ts` with a unique `value` and `slugPrefix`
2. Create a provisioning script in `scripts/` following the pattern of existing scripts
3. All events for that category must have slugs starting with the defined `slugPrefix`

Booking Widget

File: `components/cal-booker.tsx`

Uses `@calcom/embed-react`. The widget is embedded on `/book/[slug]` pages and renders the full Cal.com booking flow inline.

```
<Cal calLink={`${username}/${slug}`} config={{ layout: "month_view" }} />
```

The `slug` comes from the dynamic route param, which maps directly to the Cal.com event type slug.

Cal.com Events — Title & Price

Total: 83 events

#	Event	Price
1	3D-Hair Stroke	$600.00 USD
2	Acupuncture; 15 min (CPT-97810)	$45.00 USD
3	Acupuncture; 25 min (CPT-97811)	$75.00 USD
4	Acupuncture; 35 min (CPT-97813)	$95.00 USD
5	Acupuncture; 45 min (CPT-97814)	$110.00 USD
6	Advanced Facials and French Peel (CPT-15789)	$150.00 USD
7	Back Maintenance Cupping & Acupressure (CPT-97139)	$250.00 USD
8	Basic Gel Nail	$60.00 USD
9	Belly Button Detox	$150.00 USD
10	Bikini Waxing - Full	$75.00 USD
11	Body Transformation	$350.00 USD
12	Builder Gel	$85.00 USD
13	Chiropractic Manipulative Treatment (CPT-98943)	$125.00 USD
14	Complete Foot Treatment	$175.00 USD
15	Consultation for Campbell Spot	$250.00 USD
16	Consultation for Cherry Angioma	$250.00 USD
17	Consultation for Construction Wellness Challenge Package	$250.00 USD
18	Consultation for Large Cyst Removal	$250.00 USD
19	Consultation for Scalp Micropigmentation	$250.00 USD
20	Consultation for Skin Imperfection - Pigmentation Removal	$250.00 USD
21	Consultation for StoneIWC Cooking On-Site	$250.00 USD
22	Electrical Stimulation (CPT-97032)	$100.00 USD
23	EVS Deep Massage For Body (CPT-97124)	$350.00 USD
24	EVS Lymphatic Drainage For Body (CPT-97124)	$350.00 USD
25	Eyebrow Arch	$35.00 USD
26	Eyebrow Correction	$400.00 USD
27	Eyebrow Waxing	$35.00 USD
28	Eyelid Lower Liners (CPT-11921)	$400.00 USD
29	Eyelid Upper Liners (CPT-11921)	$400.00 USD
30	Foot Calcium Deposit Removal (CPT-29999)	$75.00 USD
31	French Manicure	$15.00 USD
32	French Peel for Face CVI (CPT-97032, CPT-15789)	$125.00 USD
33	Front Maintenance Cupping & Acupressure (CPT-97139)	$250.00 USD
34	Full Back Waxing - Full	$95.00 USD

#	Event	Price
35	Full Lash Extension & Eye Lift	$250.00 USD
36	Full Leg Waxing - Full	$95.00 USD
37	Gel Nail - Hands	$50.00 USD
38	Gel X	$90.00 USD
39	Half Back Waxing - Full	$75.00 USD
40	Half Leg Waxing - Full	$75.00 USD
41	Hand Calcium Deposit Removal (CPT-29999)	$75.00 USD
42	Head Scalp Adjustment	$75.00 USD
43	Hot/Cold-Pack (CPT-97010)	$10.00 USD
44	In Person Consultation	$250.00 USD
45	Ingrown Toe Nail Removal (CPT-11750)	$75.00 USD
46	Leakage Care for 5 (CPT-94640)	$250.00 USD
47	Lipo Treatment - Arms	$700.00 USD
48	Lipo Treatment - Buttocks	$700.00 USD
49	Lipo Treatment - Calf	$700.00 USD
50	Lipo Treatment - Face with Neck	$700.00 USD
51	Lipo Treatment - Inner Thigh	$700.00 USD
52	Lipo Treatment - Legs	$700.00 USD
53	Lipo Treatment - Stomach	$700.00 USD
54	Lipo Treatment - UnderArms	$700.00 USD
55	Lower Eyes Liner	$400.00 USD
56	Lymphatic Drainage For Body CVI (CPT-97032)	$450.00 USD
57	Lymphatic Facial Detoxification and Lift (CPT-97032)	$350.00 USD
58	Manual Microdermabrasion	$175.00 USD
59	Manual Therapy (CPT-97140)	$125.00 USD
60	Men Haircut Long & Style	$85.00 USD
61	Men Haircut Short & Style	$65.00 USD
62	Meridian Exam	$250.00 USD
63	Microdermabrasion for Body with Peel (CPT-15983)	$700.00 USD
64	Nail Removal	$15.00 USD
65	One Areola	$450.00 USD
66	One Ear Detox (CPT-69210)	$75.00 USD
67	One Side Full Body Detox - Back (CPT-97139)	$400.00 USD
68	One Side Full Body Detox - Front (CPT-97139)	$400.00 USD

#	Event	Price
69	PE-109 Full Lips	$800.00 USD
70	Refill Every 2 Weeks	$60.00 USD
71	Scalp Treatment for Hair Loss	$65.00 USD
72	Skin Fold Lymphatic Drainage for Body	$450.00 USD
73	Skin Fold Lymphatic Drainage for Face and Neck	$350.00 USD
74	Special Occasion Hair Styling	$150.00 USD
75	Stretch Adjustment (Head Scalp Adjustment)	$85.00 USD
76	TMJ (CPT-97139)	$250.00 USD
77	Two Areola	$800.00 USD
78	Two Ears Detox (CPT-69210)	$150.00 USD
79	Up Eyes Liner	$400.00 USD
80	Virtual Consultation	$3.00 USD
81	Wart Removal (CPT-17999)	$85.00 USD
82	Woman Haircut Long & Style	$95.00 USD
83	Woman Haircut Short & Style	$85.00 USD

Resend Email — Technical Documentation

Overview

Resend is the transactional email provider for Stone IWC. It is used for two email types: contact form notifications (sent to the business) and gift card delivery (sent to the gift card recipient). All email logic lives under `lib/email/`.

Environment Variables

Variable	Description
`RESEND_API_KEY`	Resend API key — required on all environments
`RESEND_FROM_EMAIL`	Sender address (e.g. `noreply@stoneiwc.com`) — must be on a verified domain
`CONTACT_EMAIL_TO`	Destination address for contact form submissions (the business inbox)

> All three must be set in Vercel environment variables for each environment (Preview, Production). The domain in `RESEND_FROM_EMAIL` must be verified in the Resend dashboard under **Domains**, otherwise all sends will fail silently.

File Structure

```
lib/email/
├── contact-template.ts    ← HTML + text templates for contact form emails
├── gift-card-template.ts  ← HTML + text templates for gift card delivery emails
└── rate-limit.ts          ← In-memory IP rate limiter (contact form only)

app/api/
├── contact/route.ts       ← POST /api/contact — sends contact form email
└── webhooks/stripe/route.ts ← Stripe webhook — sends gift card email on payment
```

Email Types

1. Contact Form Notification

Trigger: `POST /api/contact` — called when a user submits the contact form
Template: `lib/email/contact-template.ts`
Sender: `RESEND_FROM_EMAIL`
Recipient: `CONTACT_EMAIL_TO` (the business)
Reply-To: The customer's email — clicking reply in the inbox goes directly to the customer

Request body:

```
{
  firstName: string
```

```
  lastName: string
  email: string
  phone?: string   // optional
  message: string
}
```

What the email contains:

- Full name, email, phone (if provided)
- Message body with HTML-escaped content
- "Reply to [Name]" CTA button that opens a `mailto:` link

Rate limiting: 3 submissions per IP per hour (see Rate Limiting section below).

---

2. Gift Card Delivery

Trigger: `POST /api/webhooks/stripe` — fires when a `payment_intent.succeeded` event is received for a gift card purchase (`metadata.order_type === 'gift_card'`)
Template: `lib/email/gift-card-template.ts`
Sender: `RESEND_FROM_EMAIL`
Recipient: `metadata.recipient_email` (falls back to `metadata.customer_email` if not set)

What the email contains:

- Gift card value ($XX)
- The `STONE-XXXX-XXXX` redemption code
- Expiry date (1 year from purchase)
- Instructions for redeeming at checkout
- "Shop Now" CTA linking to `/products`

> The email is sent inside the Stripe webhook handler, not during the payment itself. If the webhook is not configured, no email will be sent even if the payment succeeds. See `project-documentation/gift-card.md` for webhook setup.

Rate Limiting (`lib/email/rate-limit.ts`)

Used only for the contact form endpoint to prevent abuse.

Implementation: In-memory `Map<ip, { count, resetAt }>`. Resets on cold start (serverless).

Limits:

- Window: 1 hour
- Max requests: 3 per IP per hour

How it works:

```
const { allowed, retryAfterSeconds } = checkRateLimit(ip)
if (!allowed) {
  return 429 with Retry-After header
}
```

IP detection order:

1. `x-forwarded-for` header (first IP in the list, used by Vercel/proxies)
2. `x-real-ip` header
3. Falls back to `"unknown"` (all "unknown" IPs share one bucket)

Limitation: This is an in-memory store per serverless function instance. Multiple instances do not share state. Effectively limits 3 submissions per IP per instance per hour, not globally. Acceptable for a contact form. If global enforcement is needed, replace with Upstash Redis.

Template Design

Both templates share the same design system:

- Background: `#f4f4f0` (warm off-white)
- Card background: `#ffffff`
- Header: `#1a1a1a` dark
- Accent: `#c9a96e` gold
- Body font: `Georgia, serif`
- Code font: `Courier New, monospace` (gift card only)
- Max width: 600px, centered

Templates are plain HTML strings (no React, no JSX). Each template file exports two functions:

- `*EmailHtml(data)` — full HTML string sent as the `html` field
- `*EmailText(data)` — plain text fallback sent as the `text` field

Always provide both. Email clients that block HTML fall back to plain text.

Adding a New Email Type

1. Create `lib/email/{name}-template.ts` with the data interface and two export functions (`{name}EmailHtml` , `{name}EmailText`).
2. In the API route or server action that should trigger the email, instantiate Resend and call `resend.emails.send()` :

```
import { Resend } from 'resend'
import { myEmailHtml, myEmailText } from '@/lib/email/my-template'

const resend = new Resend(process.env.RESEND_API_KEY)

const { error } = await resend.emails.send({
  from: process.env.RESEND_FROM_EMAIL!,
  to: recipientAddress,
  subject: 'Your Subject',
  html: myEmailHtml(data),
  text: myEmailText(data),
})

if (error) {
  console.error('Email send error:', error)
}
```

3. If the email is customer-facing (not internal), add `replyTo` only if appropriate.
4. If the endpoint is public-facing (user-submitted), add rate limiting using `checkRateLimit` from `lib/email/rate-limit.ts` .

Troubleshooting

Email not arriving:

1. Check that `RESEND_API_KEY` is set in the correct Vercel environment (Preview vs Production).
2. Verify the domain in `RESEND_FROM_EMAIL` is verified in the Resend dashboard under Domains.
3. Check Vercel function logs for `"Resend error:"` or `"email send error:"` entries.
4. Check the recipient's spam folder — new domains often land there.
5. For gift card emails specifically: verify the Stripe webhook is configured (see `project-documentation/gift-card.md`).

`resend.emails.send()` returns an error but the route still returns 200:

- The contact form route returns `500` on email error.
- The Stripe webhook handler logs the error but still returns `{ received: true }` — this is intentional so Stripe does not retry the webhook unnecessarily.

Rate limit hitting legitimate users:

- The contact form allows 3 submissions per IP per hour. This is intentionally strict. If it needs loosening, edit `MAX_REQUESTS` in `lib/email/rate-limit.ts`.

Webhooks — Stripe Event Handling

There is one webhook endpoint in this codebase: `POST /api/webhooks/stripe`. It handles every event Stripe sends, branching internally by `event.type` and by the PaymentIntent's `metadata.order_type`.

This doc is the reference for what that endpoint does, why, and how to debug it.

Why one endpoint for everything

Stripe lets you have multiple webhook endpoints with different event subscriptions. We deliberately use **one endpoint per environment** for simplicity:

- Easier secret management (one `STRIPE_WEBHOOK_SECRET` per scope, not many)
- One file to read when debugging
- One place that's responsible for idempotency

The cost: that handler has multiple branches. Worth it for the simplicity.

Events handled

Event	When Stripe sends it	What we do
`payment_intent.succeeded`	Card charged successfully	If storefront order → mark Sanity Order `paid` + email customer. If gift card purchase → create Sanity Gift Card + email recipient (+ purchaser). If gift card was applied → redeem balance.
`payment_intent.payment_failed`	Charge failed (decline, insufficient funds, expired card, etc.)	If storefront order → mark Sanity Order `failed`. Gift cards NOT decremented.

Any other event type is acknowledged with `{ received: true }` and ignored. Stripe is configured to only send these two — but if a future change adds more, the early return prevents accidental processing.

> **Why not `charge.succeeded`?** PaymentIntents are the higher-level abstraction; one PaymentIntent can produce multiple Charges (retries, refunds). PaymentIntent events fire at the right level for our needs.

Routing inside the handler

The handler reads `paymentIntent.metadata.order_type` to decide which flow to run:

```
event.type === 'payment_intent.succeeded'
│
├── meta.order_type === 'storefront'
│   ├── markOrderPaid(paymentIntentId)
│   └── send order confirmation email (only on pending→paid transition)
│
├── meta.gift_card_code && meta.gift_card_applied_amount > 0
│   └── redeemGiftCard(code, amount, paymentIntentId)
```

```
│
└── meta.order_type === 'gift_card'
    ├── createGiftCard(...) (idempotent by paymentIntentId)
    ├── send gift card email to recipient
    └── if recipient !== purchaser: send confirmation email to purchaser

event.type === 'payment_intent.payment_failed'
│
└── meta.order_type === 'storefront'
    └── markOrderFailed(paymentIntentId)
```

`order_type` is set when the PaymentIntent is created:

- `'storefront'` — set by `create-stripe-payment-intent/route.tsx`
- `'gift_card'` — set by `gift-cards/create-payment-intent/route.ts`

A storefront order with a gift card applied has `order_type = 'storefront'` AND `gift_card_code` set. The handler processes both: marks order paid, then redeems balance.

Signature verification

Every request is verified using `stripe.webhooks.constructEvent(body, signature, webhookSecret)`. If verification fails, the handler returns 400 immediately. This is the first thing it does — before reading the payload.

```
event = stripe.webhooks.constructEvent(body, signature, webhookSecret)
```

`webhookSecret` is `process.env.STRIPE_WEBHOOK_SECRET`. **Critical:** this must match the secret of the webhook endpoint Stripe is sending FROM. Test mode endpoint and Live mode endpoint have different secrets — see environments-and-deployments.md.

If verification fails:

- Stripe sees a 400, marks the delivery as failed, will retry on its own schedule
- The error is logged with the specific reason (`No signatures found matching the expected signature for payload`)
- Common cause: env var pointing at the wrong webhook endpoint's secret

---

Idempotency

Stripe **will** retry webhook delivery on failure. Network blips, function cold starts, intermittent errors — Stripe doesn't trust a single 200, it expects the same event to be safe to process twice. Our handler treats every event as potentially duplicated.

How each write is idempotent

Write	Idempotency guard
`markOrderPaid`	Only transitions from `pending` or `failed`; if already `paid` (or further along like `shipped`), returns `{ transitioned: false }` and skips the email send
`markOrderFailed`	Only transitions from `pending`; finalized states are not overwritten
`createGiftCard`	First fetches by `paymentIntentId`; if a Gift Card document already exists for this PI, returns its `code` without creating a duplicate
`redeemGiftCard`	Optimistic concurrency: re-fetches current balance + `redemptions` array, checks if this `paymentIntentId` is already in the array, only commits the patch if no duplicate. See `lib/gift-cards.ts`
Order confirmation email	Only sent on the `pending → paid` transition (`if (result.transitioned)`) — Stripe replays don't trigger duplicate emails
Gift card emails	Sent every time the handler runs for a gift card event, but the create step is idempotent so the email itself can be considered idempotent in practice — receiving a second email is annoying but not catastrophic

Manual test of idempotency

In any test mode purchase, after the success page loads:

1. Stripe Dashboard → Developers → Webhooks → click the endpoint → "Event deliveries"
2. Find the most recent `payment_intent.succeeded` event
3. Click "Resend"
4. Verify in Sanity: order didn't duplicate, balance didn't double-decrement

This is part of the test scenarios in runbook.md.

Always returns 200 (almost)

The handler is structured to return `{ received: true }` for any case that isn't catastrophic, even when something internal fails. This is intentional:

- Stripe retries on non-200 responses
- A persistent failure (e.g. Sanity is down) would cause Stripe to retry indefinitely
- We'd rather log the error and let an admin recover than fight retry storms

Returns that are NOT 200:

- 400 on missing `stripe-signature` header
- 400 on signature verification failure
- 500 on `Stripe is not configured` (missing env vars)
- 500 on gift card creation failure (only branch that fails the request, because the email send depends on the code)

All other errors (`markOrderPaid` failure, email send failure) are caught, logged, and the handler proceeds to return 200.

PaymentIntent metadata reference

The metadata Stripe stores on the PaymentIntent doubles as the order record visible in the Stripe Dashboard. An admin can read all of this in Stripe without opening Sanity, which matters when Sanity is down or when triaging without studio access.

Storefront PaymentIntent

Field	Example	Purpose
`order_type`	`storefront`	Branch the webhook
`order_number`	`SIWC-20260521-K2X9`	Echoed back from Sanity after order creation
`customer_email`	`jane@example.com`	Recipient of confirmation
`customer_name`	`Jane Doe`	
`customer_phone`	`+1…`	Optional
`items_count`	`3`	
`items_summary`	`1× Coffee Sampler, 2× Mug`	Truncated at 480 chars
`subtotal`	`120.00`	Pre-discount
`shipping_method`	`standard`	
`shipping_cost`	`8.00`	
`ship_name`, `ship_line1`, `ship_line2`, `ship_city`, `ship_state`, `ship_postal`, `ship_country`	Address fields, individually stored for filter/search in Stripe	
`coupon_code`, `coupon_discount`	`SUMMER10`, `12.00`	Only if a coupon was applied
`gift_card_code`, `gift_card_applied_amount`	`STONE-XXXX-XXXX`, `40.00`	Only if a gift card was applied

Gift card PaymentIntent

Field	Purpose
`order_type`	`gift_card`
`sender_name`	Required — the name that appears in the recipient email
`recipient_name`	Optional
`recipient_email`	Required — destination for the gift card delivery
`gift_card_note`	Optional personal note (max 500 chars)
`gift_card_amount`	Dollar amount as a string
`customer_email`	Purchaser's email (Stripe also has it in `receipt_email`)

> Stripe metadata values are strings (Stripe doesn't preserve numeric types). Cast back with `Number()` / `parseFloat()` on read.

Stripe webhook configuration (Dashboard)

Two endpoints, one per Stripe mode:

Stripe mode	Endpoint URL	Events
Test	`https://dev.stoneiwc.com/api/webhooks/stripe?x-vercel-protection-bypass=…&x-vercel-set-bypass-cookie=true`	`payment_intent.succeeded`, `payment_intent.payment_failed`
Live	`https://stoneiwc.com/api/webhooks/stripe`	`payment_intent.succeeded`, `payment_intent.payment_failed`

The bypass token is needed for the Test endpoint because `dev.stoneiwc.com` has Vercel Deployment Protection enabled. Production is public, no token needed.

Don't add more events without updating the handler — the route filters unknown events with an early return, so they'd just be acknowledged and dropped silently. Worse than dropping is processing the wrong way.

Debugging a webhook issue

1. **Stripe Dashboard → Developers → Webhooks → click the endpoint → "Event deliveries"** — shows all recent deliveries with response status and body
2. **Click an event** — full request/response with headers and signature
3. **Click "Resend"** — fires it again. Safe because the handler is idempotent.
4. **Vercel → Logs → filter by `/api/webhooks/stripe`** — see what the handler did with the event
5. **Stripe CLI for local debugging:**

   ```
   stripe listen --forward-to localhost:3000/api/webhooks/stripe
   stripe trigger payment_intent.succeeded
   ```

Specific failure modes are in runbook.md.

Local development

To exercise the webhook locally, forward Stripe events to your machine:

```
stripe login
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

The CLI prints a `whsec_…` signing secret. Put it in `.env.local` as `STRIPE_WEBHOOK_SECRET`. Restart `pnpm dev`.

Now any test purchase you make through `localhost:3000` will trigger real Stripe events that get forwarded to your local handler. Sanity will be written to (the same shared `production` dataset), so don't run dev test purchases with the same email as a real customer.

You can also fire arbitrary events without doing a checkout:

```
stripe trigger payment_intent.succeeded
stripe trigger payment_intent.payment_failed
```

These generate synthetic events with auto-populated metadata — they won't match real Sanity orders, so `markOrderPaid` will log `no order for PaymentIntent` but return 200. Useful for testing the signature verification path.

Adding a new event type

If you ever need to handle a new Stripe event:

1. Update the early return at the top of the handler to include the new type
2. Add a new branch after the existing ones
3. Add the event to **both** Stripe webhook endpoints (Test mode + Live mode) in the Stripe Dashboard
4. Make sure the new branch is idempotent — re-receiving the event must not double-write

Don't be tempted to subscribe to "All events" — Stripe sends a lot of housekeeping events, each one costs a function invocation, and you'd be filtering them out anyway.

See also

- orders.md — storefront order lifecycle
- gift-card.md — gift card creation and redemption details
- environments-and-deployments.md — endpoint URLs and secrets per scope
- runbook.md — webhook-specific troubleshooting

SEO — Stone IWC

This document covers everything implemented for SEO, what remains to be done, and post-deploy steps required to activate indexing.

What Was Implemented

1. Sitemap — `app/sitemap.ts`

Dynamically generated via Next.js `MetadataRoute.Sitemap`. On every build, it fetches all product slugs and article slugs from Sanity and merges them with hardcoded static routes.

Covered routes:

Route	Priority	Change Frequency
`/`	1.0	weekly
`/products`	0.9	daily
`/book`	0.9	weekly
`/education`	0.8	weekly
`/services`	0.8	monthly
`/about`	0.7	monthly
`/featured`	0.7	weekly
`/contact`	0.6	yearly
`/our-policies`	0.3	yearly
`/products/[slug]` (all)	0.8	weekly
`/education/articles/[slug]` (all)	0.7	monthly

Accessible at: `{NEXT_PUBLIC_FRONTEND_URL}/sitemap.xml`

Note: `/book/[slug]` booking pages are not included in the sitemap because they are rendered dynamically via Cal.com at runtime and should not be indexed independently.

---

2. Robots — `app/robots.ts`

Allows all crawlers on public pages and blocks:

- `/api/` — internal API routes
- `/studio/` — Sanity CMS Studio
- `/checkout/` — transactional pages
- `/view-cart/` — transactional pages

Points crawlers to the sitemap URL automatically.

---

3. Root Layout Metadata — `app/layout.tsx`

Added to the global `metadata` export:

- `metadataBase` — resolves relative URLs in child pages against `NEXT_PUBLIC_FRONTEND_URL`
- `openGraph` — type `website`, siteName, default title and description
- `twitter` — `summary_large_image` card with default title and description
- `robots` — `index: true`, `follow: true` for Google and all bots
- `alternates.canonical` — points to the base URL

---

4. JSON-LD Structured Data

A reusable `JsonLd` component lives at `components/seo/json-ld.tsx`. It renders a `<script type="application/ld+json">` tag server-side.

Schemas injected per context:

Page	Schema Type
Root layout	`LocalBusiness` + `WebSite`
`/products/[slug]`	`Product` with `Offer` (price, currency, stock status)
`/education/articles/[slug]`	`Article` with publisher and cover image

These schemas enable **rich results** in Google Search (product price/availability panels, article cards).

---

5. Dynamic Metadata — Product & Article Pages

Both `generateMetadata` functions were extended with:

- `alternates.canonical` — full URL for the specific product or article
- `openGraph` — includes Sanity image URL at 1200×630, correct `type` (`website` for products, `article` for articles), `publishedTime` for articles
- `twitter` — `summary_large_image` with Sanity image

---

6. Static Page Metadata — All Top-Level Pages

The following pages had `openGraph`, `twitter`, and `alternates.canonical` added:

`/about`, `/book`, `/services`, `/contact`, `/products`, `/education`, `/featured`

Canonical URLs use relative paths (e.g. `/about`) which Next.js resolves against `metadataBase`.

Post-Deploy Steps (Required)

Submit Sitemap to Google Search Console

This must be done once after the first production deploy.

1. Go to Google Search Console
2. Select the Stone IWC property
3. In the left menu, click **Sitemaps**
4. Enter `https://stoneiwc.com/sitemap.xml` and click **Submit**

Google will begin crawling and indexing all routes listed in the sitemap.

Verify Sitemap and Robots Are Live

After deploy, manually verify:

- `https://stoneiwc.com/sitemap.xml` — should return XML with all routes
- `https://stoneiwc.com/robots.txt` — should list `Disallow` rules and `Sitemap:` pointer

Remaining Work (Not Yet Implemented)

OpenGraph Default Image

Currently no fallback image exists for social previews on pages that do not have a Sanity image (homepage, services, contact, etc.). When shared on social media, these pages will show no preview image.

To fix: Add `app/opengraph-image.tsx` (or a static `app/opengraph-image.png`) using the Next.js `ImageResponse` API or a pre-designed static asset.

Booking Pages (`/book/[slug]`)

Individual booking pages have a `generateMetadata` function pulling title and description from Cal.com, but they are missing:

- `openGraph` and `twitter` metadata
- `alternates.canonical`
- JSON-LD (`Service` schema would be appropriate here)

These pages also have no `generateStaticParams`, so they are fully dynamic (not SSG). Adding `generateStaticParams` using `getEventTypes()` would make them statically generated and improve crawl performance.

Homepage (`/`)

The root `page.tsx` has no `metadata` export of its own. It inherits the root layout defaults, which is acceptable but not ideal. A dedicated homepage metadata with a richer description and a targeted `openGraph` image would improve click-through rates from search results.

`keywords` Meta Tag

Not implemented. Google officially ignores the `keywords` meta tag, so this is low priority. Bing still reads it. If Bing coverage matters, add a `keywords` array to the static page metadata exports.

Structured Data — BreadcrumbList

No breadcrumb schema is in place. Adding `BreadcrumbList` JSON-LD to product and article pages would enable breadcrumb rich results in Google Search, which increase CTR.

Structured Data — FAQPage

If any page includes a Q&A or FAQ section (e.g. services, education), wrapping those items in a `FAQPage` schema would unlock FAQ rich results in Google.

International / Multi-language

Not applicable. The site is English-only. If a Spanish version is ever added, `hreflang` alternates will be needed in the root layout metadata.

Key Files

File	Purpose
`app/sitemap.ts`	Dynamic sitemap generation
`app/robots.ts`	Crawler rules
`app/layout.tsx`	Root metadata, JSON-LD Organization/WebSite
`components/seo/json-ld.tsx`	Reusable JSON-LD script component
`app/products/[slug]/page.tsx`	Product metadata + JSON-LD
`app/education/articles/[slug]/page.tsx`	Article metadata + JSON-LD

Content Workflows — Studio User Guide

This doc is for **anyone managing content** (staff, admins, content editors) — not just developers. It walks through the tasks you do in Sanity Studio: adding products, services, images, managing orders and gift cards, editing coupons and shipping.

The Studio URL: ask your admin. It's typically `https://stoneiwc-studio.sanity.studio` (or whatever name was chosen during deploy). You also need a Sanity account with access to the project.

For developer-oriented schema details (field validations, GROQ queries), see sanity-cms.md.

Contents

Getting started in Studio

If this is your first time, do these things in order before touching anything else.

1. Get access

You need a Sanity account that's been added to the Stone IWC project. Steps:

1. Your admin sends you an invite by email — subject like "You've been invited to a Sanity project"
2. Open the email, click the invite link
3. Sign in or create a Sanity account using the **same email** the invite was sent to
4. Once accepted, the Stone IWC project shows up in your Sanity dashboard

> If the invite link is expired or missing, ask the admin to resend it. Don't sign up with a different email — you'll create a separate account that has no access.

2. Open the Studio

Two ways to reach it:

- **Hosted URL** (recommended for daily work) — the link the admin gave you, looks like `https://stoneiwc-studio.sanity.studio`. Bookmark it.
- **Local** (only if you're working with a developer running the project on their machine) — `http://localhost:3333`

Sign in with the same Sanity account from step 1.

3. Understand the sidebar

The left sidebar is the **map of everything you can edit**. The top-level sections are:

Section	Contains
Pages	Editable images, text, and assets for each public page of the site (Home, About Us, Services, Education, Featured On)
Products	The storefront — All products and Categories
Order	Customer orders. Day-to-day fulfillment work happens here.
Gift Cards	Issued + manually created gift cards
Shipping & Coupons	Discount codes and shipping options shown at checkout

Within **Pages**, the layout mirrors the site navigation. So **Pages** → **About Us** → **Our Story** → **Images** edits the images on the `/about/our-story` URL.

4. Drafts vs Published — the most important thing to know

Every document in Studio has two states:

- **Draft** (the local copy you're editing — orange dot in the corner of the field)
- **Published** (the version live on the website)

When you type, Sanity **auto-saves** your draft every few seconds. **But that does NOT make it live.** To make changes visible on `stoneiwc.com`, you must click the green **Publish** button at the bottom of the document.

```
You type → auto-saved as draft (visitors still see the old published version)
You click Publish → live within ~60 seconds (ISR cache refresh)
```

If you don't see your change on the live site after a minute:

1. Did you click Publish? Look for the green button — when there's an unpublished draft, it says "Publish" and is highlighted
2. Hard-refresh the page (Cmd+Shift+R on Mac, Ctrl+Shift+R on Windows)
3. Wait another 60 seconds — ISR caches refresh on the next request after the timeout

To **discard** a draft without publishing: open the document → three-dot menu → "Discard changes".

5. Undo / revision history

Sanity tracks every change. If you mess something up:

1. Open the document
2. Click the **clock icon** in the top-right corner (or the document's three-dot menu → "Inspect")
3. Browse the timeline — you can see who changed what, when
4. Click any past version to preview it, or restore to that version

This is your safety net. You almost can't permanently break anything as long as you don't *delete* the document.

6. Search — Cmd+K

Press **Cmd+K** (Mac) or **Ctrl+K** (Windows) anywhere in Studio to open the global search. Type a product name, an order number, a customer email, anything — Studio finds matching documents across all types. Useful when there are too many products to scroll through.

7. Image hotspot — what it does

When you upload an image, click the image to edit it and you'll see a circle you can drag — the **hotspot**. This tells Studio which part of the image is the "focal point" — the part that must stay visible when the image gets cropped for different layouts (square card vs. wide banner).

Practical tip:

- For a portrait → drag the hotspot to the face
- For a wide scene → drag the hotspot to the main subject
- For a flat-lay (e.g. product photography) → center the hotspot

Also fill in the **alt text** — read by screen readers, used by Google. A short description of what's in the image. Not the file name.

8. If something looks wrong

- "I can't see my change on the site" → see Drafts vs Published above
- "I accidentally deleted something" → revision history (#5) — if the document still exists, restore it. If you fully deleted the document, ask the admin (they may be able to recover from a Sanity dataset backup)
- "I see fields I don't recognize" → don't fill them out randomly; ask the admin. New schema fields sometimes appear without instructions
- "Studio is showing 'Insufficient permissions'" → you have Viewer access, not Editor. Ask the admin to upgrade you

Quick reference

Action	How
Save a draft	Automatic, no button needed
Publish (make live)	Green "Publish" button at the bottom
Discard a draft	Three-dot menu → Discard changes
Restore an older version	Clock icon → pick version → Restore
Search anything	Cmd+K / Ctrl+K
Switch documents quickly	Click in the sidebar, the breadcrumbs stay

Once you've done all this once, the workflows in the rest of this doc make sense.

Adding a product

Sidebar: **Products** → **All Products** → **Create**

Required fields

Field	What to enter
Name	The product's display name. Title-case.
Slug	Auto-generates from name (e.g. "Coffee Sampler" → `coffee-sampler`). Used in URL `/products/coffee-sampler`. Change only before first publish; changing later breaks customer bookmarks and SEO.
Price	Dollar amount as a number. **No `$` sign.** Decimal allowed (`24.99`).
Short description	One sentence (max 200 characters). Shown on product cards.
Description	Full text. Shown on the product detail page.
Image	See Uploading images below
Category	Must already exist (Products → Categories). Pick from the dropdown.

Optional fields

Field	What to enter
Original price	Set this only if the product is on sale. The card shows the original price crossed out and a "Sale" badge.
Tags	Free-form. Common: `bestseller`, `organic`, `new`, `gift-set`. Used for filtering.
In stock	Default `true`. Set to `false` to hide the "Add to cart" button (product still visible).
Featured	If `true`, shows on the homepage featured carousel.

After saving

The product is live within ~60 seconds (ISR cache refresh) at `https://stoneiwc.com/products/<slug>`. If you don't see it after a minute, force a hard refresh (Ctrl+Shift+R / Cmd+Shift+R) or wait another cycle.

Do not do this

- **Don't change the slug** after publishing. If you must, set up a redirect manually (file an issue with the dev team).
- **Don't delete a product** that has orders referencing it — the order page will show "Item" as a placeholder name. Better to set `inStock: false`.
- **Don't rename the gift card product slug** (`stoneiwc-gift-certificate`). It's hardcoded as the trigger for the special purchase UI.

Adding a service / Cal.com booking

Services that customers can book (consultations, programs, etc.) are **NOT** in Sanity. They live in **Cal.com**. The storefront just reads from Cal.com's API and renders the booking UI.

To add a new bookable service

1. **Create the event in Cal.com:**
    - Cal.com → Event Types → Create
    - Slug — use kebab-case (e.g. `wellness-consultation`)
    - Title, description, duration, location (in-person / Zoom / etc.), price (if paid)
    - Save

2. **Slug conventions** (important for how it's displayed on the storefront):

Slug pattern	Effect
`*-free` (e.g. `intro-call-free`)	Card shows "Free"
`*-consultation-*` (e.g. `health-consultation-30min`)	Card shows "Price for Consultation" badge
Anything else	Shows the actual Cal.com price

3. The new service should appear at `https://stoneiwc.com/book/<slug>` within seconds (Cal.com data isn't ISR-cached the same way).

4. **Provisioning via script** (developer task): if you have many services to create at once, the dev team can use `scripts/cal_events_*.py` to provision them in bulk. See cal-events.md.

Editing or removing a service

- Edit in Cal.com directly. Don't touch Sanity for services.
- Removing: delete in Cal.com. The storefront will stop showing it. Existing bookings remain in Cal.com history.

See cal-com-booking.md for the technical details.

Uploading images correctly

Image quality is a recurring pain point. Following these guidelines avoids 90% of issues.

Recommended dimensions

Use	Min dimensions	Aspect ratio	Format
Product image	1200 × 1200	1:1 (square)	JPG or PNG
Hero slide	2400 × 1200	2:1 (landscape)	JPG
Team member portrait	800 × 1000	4:5 (portrait)	JPG
Article cover	1600 × 900	16:9 (landscape)	JPG
Page section image	At least 1600 wide	Varies	JPG

Sanity's CDN will resize down — uploading larger is fine, uploading smaller looks blurry. **Don't upload images larger than ~5MB** — it works but slows the Studio.

The alt text

Every image field has an `alt` text input (sometimes labeled "Alternative text" or hidden under "Edit details"). **Fill it in.** This:

- Is read aloud by screen readers (accessibility)
- Is used by Google to understand the image (SEO)
- Falls back when an image fails to load

A good alt text describes the image, not the file: ✓ "Stone-built spa entrance with cedar door" vs. ✗ "image1.jpg" or "Spa".

The hotspot

Sanity images support a "hotspot" — a focal point used when the image is cropped. Click the image in the Studio, drag the hotspot circle to the part that must stay visible. Useful for portraits (keep the face centered when the card is square) and wide shots (keep the subject in frame when cropped to vertical).

Managing orders (fulfillment)

Sidebar: **Order**

The order list shows the most recent orders first. Each row shows: order number (`SIWC-…`), status badge, total amount, customer email.

The order lifecycle

```
pending  →  paid  →  shipped  →  delivered
                                  refunded (terminal)
                                  cancelled (terminal)
            failed (terminal)
```

- **`pending`** — order created, payment in flight. Should advance to `paid` within seconds. If it stays `pending` longer than 1 minute, check the runbook.
- **`paid`** — payment succeeded. Time to fulfill.
- **`shipped`** — package handed to carrier. Fill `shippedAt` , `trackingCarrier` , `trackingNumber` .
- **`delivered`** — confirmed received (optional; some teams skip this).
- **`refunded`** — money returned to customer. Issue the refund in Stripe Dashboard FIRST, then set this status in Sanity.
- **`cancelled`** — order cancelled before fulfillment. Use `internalNotes` to explain why.
- **`failed`** — payment failed at checkout. No money was charged. No action needed.

To ship an order

1. Open the order
2. Fill in:
   - `Shipped at` → current date+time
   - `Tracking carrier` → e.g. `USPS` , `UPS` , `FedEx`
   - `Tracking number` → from the carrier's label
3. Change `status` to `shipped`
4. Publish (the green button)

Customer doesn't get an automated "your order has shipped" email yet — this is a future enhancement (see open-backlog.md). For now, send an email manually if your process requires it.

To refund an order

1. **Stripe Dashboard** → **Payments** → find the payment by amount or customer email → click → "Refund"
2. **Sanity Studio** → **Orders** → open the order → set `status` to `refunded`
3. Add a note in `internalNotes` explaining the reason

If only refunding partially: do the partial refund in Stripe, then in Sanity status stays at the previous state (e.g. `delivered`) with a note about the partial refund. The Sanity `total` field is NOT auto-updated; record the refund amount in `internalNotes` .

Internal notes

The `internalNotes` field is freeform text. Use it for:

- "Customer asked for back-door delivery"
- "Reshipped after first package lost in transit"
- "Refunded $20 for late delivery, kept the rest"

These notes are not visible to the customer.

Managing gift cards

Sidebar: **Gift Cards**

Gift cards are created automatically when someone buys one through the storefront. You'll only manage them when:

- A customer reports a problem (lost code, partially used, etc.)
- You want to manually issue a card (for promotions, replacements)
- You need to deactivate a card

Fields

Field	Notes
`code`	Customer-facing code, format `STONE-XXXX-XXXX`. Auto-generated, don't change.
`originalAmount`	What the card was sold for. Don't change after creation.
`currentBalance`	What's left. Decreases on each redemption.
`status`	`active` (usable) / `redeemed` (fully consumed) / `expired` (past expiry) / `void` (admin-revoked)
`purchaser`	Who bought it (name + email)
`recipient`	Who it's for (name + email, may equal purchaser)
`note`	Personal message from purchaser
`expiresAt`	Default: 1 year from purchase
`redemptions[]`	Array of redemption events — each has `amount`, `orderNumber`, `paymentIntentId`, `redeemedAt`
`paymentIntentId`	Stripe PaymentIntent that paid for the card (used for idempotency)

To manually issue a gift card

(For promotions, refunds in card form, etc.)

1. Sidebar **Gift Cards** → **Create**
2. Fill:
   - `code` — generate one yourself or copy the format `STONE-XXXX-XXXX`. Must be unique.
   - `originalAmount` — dollar amount
   - `currentBalance` — same as original
   - `status` — `active`
   - `purchaser` — your team's contact (e.g. info@stoneiwc.com)
   - `recipient` — customer's name + email
   - `expiresAt` — pick a date (default 1 year)
   - Leave `paymentIntentId` blank (no Stripe PI for manual issuance)
3. Publish
4. Email the code to the recipient (no automatic email is sent for manually-created cards)

To deactivate a card (lost/stolen/dispute)

Set `status` to `void`. The validation API will reject it on next use.

To resend the gift card email

There's no UI button for this. Currently the only way is to manually compose an email with the code. Future enhancement: a "Resend email" button (see open-backlog.md).

See also

- gift-card.md — technical details on creation, validation, redemption

---

Editing coupons

Sidebar: **Shipping & Coupons** → **Coupons**

To create a coupon

1. Create
2. Fill:
   - `code` — case-sensitive, what customer types (e.g. `SUMMER10`, `WELCOME`)
   - `description` — internal only, won't be shown to customers (e.g. "Summer 2026 promo")
   - `type` — `percentage` (off subtotal) or `fixed` (dollar amount off)
   - `value` — for percentage, enter `10` for 10%. For fixed, enter `10` for $10.
   - `minSubtotal` — optional. e.g. `50` means coupon only valid on orders ≥ $50.
   - `isActive` — `true` to enable
3. Publish

The coupon is usable immediately (no ISR delay — coupon validation is a server fetch on each checkout).

To disable a coupon

Set `isActive` to `false`. The validation API will reject it.

Coupon behavior

- Coupons apply to **merchandise subtotal**, before shipping
- A coupon and a gift card can be combined (see orders.md)
- Discount is capped at the subtotal (a $100 coupon on a $50 cart = $50 off, not a $50 refund)
- One coupon per order (customers can't stack coupons)

Editing shipping methods

Sidebar: **Shipping & Coupons** → **Shipping Methods**

To add a shipping method

1. Create
2. Fill:

- `id` — slug used in Stripe metadata (e.g. `standard`, `express`, `local-pickup`)
- `name` — display name at checkout (e.g. "Standard Shipping")
- `description` — shown under the name (e.g. "5-7 business days")
- `cost` — dollar amount. **Use `0` for free shipping, never below `0.50`** (Stripe minimum charge is $0.50 — see warning below)
- `isActive` — `true` to show at checkout
- `order` — display order, lower numbers first

3. Publish

Warning: never set cost to a non-zero amount below $0.50

Stripe rejects charges below $0.50. If a customer's order total falls below this (e.g. small product + $0.10 shipping with a gift card covering the rest), the PaymentIntent fails. Either set shipping to $0 (free) or ≥ $0.50.

There's no Sanity validation enforcing this yet. Be careful.

To disable a method

Set `isActive` to `false`. It won't appear at checkout.

Editing page images and hero slides

Sidebar: **Pages** → **[section]** → **Images** (varies by section)

Many pages have a singleton "Images" document with multiple slots (hero, secondary, gallery). You edit them in place — no create/delete needed.

Homepage hero slides

Sidebar: **Pages** → **Home** → **Hero Slides**

These are ordered — drag to reorder, or set `orderRank` numerically. Each slide has:

- `title`, `subtitle`, `description`
- `image` (with alt text)
- (Optionally a CTA button)

Other page images

Each page has its own image set. The Studio sidebar mirrors the site nav — **Pages** → **About Us** → **Our Story** → **Images**, etc. See sanity-cms.md for the full list.

Publishing articles

Sidebar: **Pages** → **Education** → **Articles**

1. Create
2. Fill:
   - `title`, `slug` (URL becomes `/education/articles/<slug>`)
   - `publishedAt` — set to current time or schedule for later
   - `coverImage` — recommended 1600 × 900
   - `excerpt` — one paragraph teaser

    - `tags` — for categorization on the index page
    - `body` — rich text editor (block content). Headings, paragraphs, images, links all supported.
3. Publish

Article appears on `/education` within ~60 seconds.

Updating team members and partners

Sidebar: **Pages** → **About Us** → **Team Members** or **Partners & Affiliates**

Both are ordered lists with similar fields:

- `name`, `title`, `image`, `bio`
- (Team) `specialties` — array of strings
- (Partner) `websiteUrl` — clickable on the partners section

Drag to reorder or set `orderRank` numerically.

A note on publishing

In Sanity Studio, **clicking "Publish" makes changes go live**. Saving (the auto-save dot in the corner) only saves the draft — readers on the live site won't see it until you publish.

Always publish your changes. Drafts can pile up unnoticed.

See also

- sanity-cms.md — schema fields and technical structure
- orders.md — order lifecycle in depth
- gift-card.md — gift card flow in depth
- cal-com-booking.md — service / booking specifics
- open-backlog.md — known limitations / planned improvements

Frontend & UI

Conventions for building and modifying the user interface. Read this before adding pages, components, or touching theming.

Stack

Layer	Tool
Framework	Next.js 16 App Router (TypeScript)
Styling	Tailwind CSS
Components	shadcn/ui (Radix UI primitives, copied into `components/ui/`)
Icons	`lucide-react`
Fonts	Cormorant Garamond (display) + Lato (body), via `next/font/google`

Design tokens

All colors are defined as CSS custom properties in `app/globals.css` and exposed as Tailwind utilities via `tailwind.config.ts`. **Use the token, not raw hex.**

Tailwind class	Token	Use
`bg-background` / `text-foreground`	base	Page background, default text
`bg-card` / `text-card-foreground`	card	Card surfaces
`bg-primary` / `text-primary-foreground`	primary (gold)	CTAs, key brand color (`#c9a96e`-ish gold)
`bg-secondary` / `text-secondary-foreground`	secondary	Subdued surfaces
`bg-muted` / `text-muted-foreground`	muted	Background for inactive states; muted body text
`bg-accent` / `text-accent-foreground`	accent (deep brown)	Hover surfaces, secondary CTAs
`bg-destructive` / `text-destructive-foreground`	destructive (red)	Errors, danger actions
`border-border`	border	All borders. Avoid `border-gray-*`.

> **Dark mode is configured** (`darkMode: ['class']`) but not actively used. The `:root` CSS vars and the `.dark` variant are both defined. If you want to enable a toggle, add a `ThemeProvider` consumer + `next-themes`.

Typography

Two font families, loaded once in `app/layout.tsx`:

Class	Family	Use
`font-sans`	Cormorant Garamond	Display: headings, key product copy, navbar, footer brand
`font-body`	Lato	Body copy: paragraphs, form labels, captions, button text

```
<h1 className="font-sans text-3xl font-semibold tracking-wide">Page Title</h1>
<p className="font-body text-base text-muted-foreground">Body copy here.</p>
```

Default body font is `font-body` (set on `<body>` in the root layout) — so on plain text you don't need to add it. Add `font-sans` explicitly for headings and brand copy.

Letter-spacing conventions:

- Major headings: `tracking-wide` (slightly looser, feels editorial)
- Small caps / labels: `tracking-[2-3px]` (e.g. `letter-spacing: 3px`) for the "STONE INTERNATIONAL WELLNESS CENTER" style chrome

---

Component library

shadcn/ui (`components/ui/`)

Most low-level components — Button, Input, Select, Sheet, Dialog, etc. — come from shadcn. These are **copied source files**, not a dependency. To upgrade or add new ones:

```
npx shadcn@latest add <component>
```

They install into `components/ui/`. Once added, edit freely — they're your code now.

> Don't manually edit core Tailwind tokens to change shadcn behavior. Edit the token in `globals.css` (e.g. `--primary`) and every shadcn component re-themes automatically.

Project-specific components (`components/`)

Composites built on top of shadcn:

- **`navbar.tsx`** / **`mobile-nav.tsx`** — top navigation
- **`footer.tsx`** — site-wide footer with newsletter form
- **`page-header.tsx`** — repeated page title + subtitle pattern
- **`cart-sheet.tsx`** — slide-over cart (Sheet primitive)
- **`cal-booker.tsx`** — wrapper around `@calcom/embed-react`
- **`checkout/SelfCheckoutSection.tsx`** + **`CheckoutDetailsSection.tsx`** — the two-column checkout layout
- **`products/`** — listing grid, filters, product card, gift card purchase
- **`home/`**, **`education/`**, **`contact/`** — page-specific sections
- **`seo/`** — JSON-LD structured data components

Page layout pattern

Most content pages follow this structure:

```
export default function MyPage() {
  return (
    <>
      <PageHeader title="My Page" subtitle="Short description of what this is." />

      <section className="py-14 lg:py-20">
        <div className="mx-auto max-w-6xl px-6">
          {/* content */}
        </div>
      </section>
    </>
  )
}
```

Notes:

- `<>...</>` Fragment instead of a wrapper div — `Navbar` and `Footer` are in the root layout
- `py-14 lg:py-20` — standard vertical rhythm
- `mx-auto max-w-6xl px-6` — standard centered container. Use `max-w-7xl` for wider content (product listing), `max-w-3xl` for prose (articles)

Adding a new page

1. Create `app/<route>/page.tsx`
2. Use the layout pattern above
3. If the page needs Sanity data: server-fetch in the component (it's a Server Component by default) using helpers from `lib/sanity.queries.ts`
4. If using ISR (recommended for any Sanity-backed page):

   ```
   export const revalidate = 60
   ```

5. If the page needs metadata for SEO, export `metadata`:

   ```
   import type { Metadata } from 'next'
   export const metadata: Metadata = {
     title: 'My Page — Stone IWC',
     description: '...',
   }
   ```

Dynamic routes (`[slug]`)

For `/products/[slug]`, `/book/[slug]`, etc.:

```
export async function generateStaticParams() {
  const slugs = await getAllProductSlugs()
  return slugs.map((slug) => ({ slug }))
}

export async function generateMetadata({ params }): Promise<Metadata> {
  const product = await getProductBySlug(params.slug)
  return {
    title: `${product.name} — Stone IWC`,
    description: product.shortDescription,
  }
}
```

```
export default async function ProductDetailPage({ params }) {
  const product = await getProductBySlug(params.slug)
  if (!product) return notFound()
  // ...
}
```

`generateStaticParams` pre-builds all known slugs at build time; `revalidate = 60` keeps them fresh.

Client components — when to use

Default to Server Components. Add `'use client'` only when you need:

- React state (`useState` , `useReducer`)
- Effects (`useEffect`)
- Browser APIs (localStorage, window, etc.)
- Event handlers (`onClick` , `onChange` — these don't work in Server Components)
- Third-party libraries that use any of the above

Examples of client components in this repo:

- `lib/cart-context.tsx` — uses `useReducer` + `localStorage`
- `components/checkout/SelfCheckoutSection.tsx` — form state, Stripe Elements
- `components/cal-booker.tsx` — Cal.com embed needs window

Server components can render client components but not vice versa (a client component can render server components only when passed as `children`).

ISR and revalidation

All Sanity-backed pages should have:

```
export const revalidate = 60   // seconds
```

Combined with tagged queries in `lib/sanity.queries.ts` :

```
client.fetch(query, params, { next: { tags: ['product'] } })
```

On-demand revalidation by tag: `POST /api/revalidate` with `{ tag: 'product' }` . Sanity webhooks can call this so changes propagate immediately. Currently not wired up by default — see open-backlog.md.

Forms

There's no form library — forms use raw React state. Two examples to follow:

- `app/contact/page.tsx` — simple submission, server validation
- `components/checkout/SelfCheckoutSection.tsx` — complex multi-step

Conventions:

- State as a single `form` object with `setForm((f) => ({ ...f, field: value }))`
- Client-side validation in an `isFormValid()` helper for button disabled state

- Server-side validation always re-checks (never trust client)
- Submit button shows loading state, becomes disabled while in flight
- Errors shown in a `<div className="rounded-sm border border-red-200 bg-red-50 p-4">` block

For inputs, use shadcn `<Input>` / `<Textarea>` / `<Select>` from `components/ui/`.

Cart state

The cart is a React context defined in `lib/cart-context.tsx` and provided from the root layout. It exposes:

```
const {
  items, totalItems, subtotal,
  appliedCoupon, applyCoupon, removeCoupon, discountAmount,
  totalPrice,                          // subtotal - coupon (no shipping/gift card)
  addItem, removeItem, updateQuantity, clearCart,
  isOpen, setOpen,                     // cart sheet visibility
} = useCart()
```

Persistence: items and applied coupon are saved to `localStorage` under the key `stone-iwc-cart`. Loaded on mount; saved on every change.

> The checkout page computes its own `finalTotal` because it has access to shipping and gift card discounts which the cart context doesn't know about. See `app/checkout/page.tsx`.

Images — Next.js + Sanity

For Sanity-managed images:

```
import Image from 'next/image'
import { urlFor } from '@/lib/sanity.image'

<Image
  src={urlFor(product.image).width(800).height(800).url()}
  alt={product.image.alt || product.name}
  width={800}
  height={800}
  className="object-cover"
/>
```

Always:

- Pass `width` and `height` (or `fill` for filling a parent) — Next.js needs them for layout shift prevention
- Provide an `alt` that's a real description, not empty
- Use `urlFor()` with explicit dimensions to avoid serving multi-MB originals

For static images (logos, icons, decorative):

```
<Image src="/logo.png" alt="Stone IWC" width={200} height={60} />
```

Place them in `public/`.

Animation and motion

There's no animation library in this project — no Framer Motion, no GSAP. The visual style is intentionally restrained: smooth scrolling (`scroll-behavior: smooth`), `transition-colors` on hover, occasional `animate-spin` for loaders. Adding heavy motion is a deliberate design choice; align with the team before adopting.

Responsive breakpoints

Standard Tailwind:

Prefix	Min width	Use
(none)	0	Mobile first; default classes are mobile
`sm:`	640px	Small tablets
`md:`	768px	Tablets
`lg:`	1024px	Desktops
`xl:`	1280px	Wide desktops

The mobile-bar pattern (e.g. `components/booking-mobile-bar.tsx`) is `block lg:hidden` — visible on mobile, hidden on desktop. Conversely, sidebars are `hidden lg:block` .

Accessibility checklist

Things to remember:

- Every `<Image>` has a non-empty `alt`
- Form inputs have associated `<label>` (or `aria-label`)
- Interactive non-button elements get `role="button"` + `tabIndex={0}` + key handlers
- Color contrast: the primary gold on white can be borderline — use `text-foreground` for body text, gold only for emphasis
- Modals and Sheets from Radix already handle focus trapping correctly — don't fight the library

Run a quick check with the **Accessibility tab** in Chrome DevTools or `pnpm dlx lighthouse https://localhost:3000` before shipping a major UI change.

SEO — quick reference

Details in seo.md. Essentials when adding/editing a page:

- Every page exports `metadata` (or `generateMetadata` for dynamic routes) with `title` and `description`
- For Open Graph: extend `metadata.openGraph` if the page is shareable
- For product pages: the JSON-LD `<Product>` component is in `components/seo/` — include it on detail pages
- Sitemap and robots are auto-generated from `app/sitemap.ts` and `app/robots.ts` — add new public routes there if they don't have a Sanity slug

---

A few patterns to follow

Goal	Pattern
Centered container	`mx-auto max-w-6xl px-6`
Card	`rounded-sm border border-border bg-card p-6`
Primary button	`rounded-sm bg-primary px-6 py-3 text-sm font-body font-bold tracking-wider text-primary-foreground transition-colors hover:bg-primary/90`
Secondary button	Same shape + `bg-secondary hover:bg-secondary/80`
Section spacing	`py-14 lg:py-20` for major sections, `py-8 lg:py-12` for sub-sections
Form input	`rounded-sm border border-input bg-background px-3 py-2 text-sm font-body outline-none focus:border-primary`

See also

- architecture.md — system overview, where the frontend fits
- sanity-cms.md — data layer that powers most pages
- seo.md — metadata, sitemap, structured data
- content-workflows.md — how Studio edits propagate to UI

Security & Compliance

A short, practical checklist of the security-relevant choices in this codebase and what to keep doing right. Not a substitute for a proper penetration test, but a guard against the easy mistakes.

Threat model in one paragraph

Stone IWC is a small e-commerce site with **no customer accounts** — there are no user passwords, sessions, or PII beyond what's collected at checkout. The blast radius of a compromise is mostly limited to: defrauded orders, customer email/address exposure, and gift card fraud. The most valuable secrets are the **Stripe live secret key** and the **Sanity API token** with Editor permission; the rest are recoverable.

Secrets — what they are, where they live

Secret	Lives in	If leaked, attacker can
`STRIPE_SECRET_KEY` (live)	Vercel env (Production scope)	Charge cards, issue refunds, view PII, view full Stripe dashboard
`STRIPE_SECRET_KEY` (test)	Vercel env (Preview/Dev), local `.env.local`	Charge fake cards (no real impact, but spam Stripe events)
`STRIPE_WEBHOOK_SECRET`	Vercel env	Forge webhook events — impersonate Stripe to mark orders paid without paying
`SANITY_API_TOKEN` (Editor)	Vercel env, local `.env.local`	Write to Sanity: create/modify orders, alter products, drain gift cards
`RESEND_API_KEY`	Vercel env, local `.env.local`	Send emails from `info@stoneiwc.com`. Reputational damage if used for spam/phishing
`CAL_API_KEY`	Vercel env, local `.env.local`	Read bookings, modify event types
`NEXT_PUBLIC_*`	Public on the client	Not secrets — these are visible in the browser bundle. Don't put secrets behind this prefix.

Rules:

- **Never commit `.env.local`** — it's gitignored, keep it that way
- **Never put a secret behind `NEXT_PUBLIC_*`** — the build will inline it into the client bundle
- **Don't paste secrets into chat, Linear, GitHub issues, or screenshots** — they end up indexed
- **Rotate immediately** if you suspect leakage

Rotating a secret

Stripe keys

1. Stripe Dashboard → Developers → API keys → "Reveal and roll" on the leaked key
2. Vercel → update the `STRIPE_SECRET_KEY` for the affected scope
3. Redeploy
4. Stripe revokes the old key automatically after a short grace period

Stripe webhook secret

You can't rotate the secret without re-creating the endpoint (Stripe doesn't expose a "rotate" button for it):

1. Stripe → Developers → Webhooks → delete the leaked endpoint
2. Create a new one with the same URL + events
3. Copy the new signing secret
4. Update Vercel `STRIPE_WEBHOOK_SECRET` and redeploy

Sanity token

1. Sanity Manage → Project → API → Tokens → revoke the leaked token
2. Create a new one with the same permission level (Editor)
3. Update `SANITY_API_TOKEN` in Vercel + local `.env.local`
4. Redeploy

Resend key

1. Resend Dashboard → API Keys → revoke
2. Create new
3. Update `RESEND_API_KEY` in Vercel and redeploy

Payment security (Stripe)

Concern	How we handle it
Card data never touches our server	Stripe Elements collects card details in an iframe sandboxed to Stripe's domain. Only the `paymentMethod` ID (a token) crosses our backend. We are out of PCI scope.
Charge amount tampering	The PaymentIntent amount is created server-side, but the server currently trusts the client-sent `totalAmount`. **Future hardening:** re-fetch product prices from Sanity at server time and validate before creating the PI. See open-backlog.md.
Webhook impersonation	Every webhook payload is signature-verified with `stripe.webhooks.constructEvent` using `STRIPE_WEBHOOK_SECRET`. A request without a valid signature returns 400 immediately.
Idempotency	Webhook handler is idempotent by `stripePaymentIntentId` — replays don't double-charge / double-redeem. See webhooks.md.
Refund process	Refunds happen in Stripe Dashboard manually. Sanity Order status is updated after, by hand. No automated refund API exposed (so no API endpoint to abuse).

Customer PII

What we collect and where it lives:

Data	Stored in	Retention
Name, email, phone, shipping/billing address	Sanity Order document; Stripe PaymentIntent metadata	Indefinite (no auto-deletion)
Email (for contact form)	Resend logs (~30 days), team inbox	Indefinite if not deleted
Email (for newsletter, if implemented)	Not currently implemented	N/A
Card data	**Never our server.** Stripe holds the payment method.	Stripe's retention applies
IP address	Used for rate-limiting contact form. Not persisted.	Ephemeral

If a customer requests deletion (GDPR / similar):

1. Sanity → Orders → find by email → delete documents (or anonymize: replace email and name with `redacted@`)
2. Stripe → Customers → find by email → delete customer (this also detaches PIs from the customer; the PI itself stays for accounting)
3. Resend → Logs → no built-in delete; logs auto-expire

There is no formal data deletion API. Honor requests manually.

Sanity access control

- The `SANITY_API_TOKEN` used at runtime has **Editor** permission — needed to create/update orders + gift cards
- Other tokens (e.g. for dev tools, Sanity Vision queries) should be **Viewer** unless they need to write
- Sanity Studio access is per-user — managed in Sanity Manage → Members. Don't share login credentials.
- The frontend never reads/writes Sanity directly; all access goes through API routes that run server-side. This is the reason `SANITY_API_TOKEN` is *not* prefixed `NEXT_PUBLIC_*`.

Rate limiting

Endpoint	Limit	Where
`POST /api/contact`	Per-IP, see `lib/email/rate-limit.ts`	In-memory (resets on function cold start)
Other endpoints	None	Vercel's built-in DDoS protection at the platform level

The in-memory rate limit resets on every cold start, so a determined attacker could bypass it. For higher confidence, move to Vercel Edge Config / Upstash Redis. Acceptable today because the cost of contact form spam is just team inbox noise.

Webhook URLs and public exposure

- Webhook URL: `https://stoneiwc.com/api/webhooks/stripe` — **publicly accessible by design.** Stripe must be able to reach it.
- Protection: every request is signature-verified. Unsigned/wrongly-signed requests get 400.
- Don't add basic auth or Vercel SSO to production — it would block Stripe.
- The preview webhook URL includes a Vercel protection bypass token: `?x-vercel-protection-bypass=…`. **This token is itself a secret.** Treat the Stripe webhook URL as confidential — don't post screenshots of it.

Dependency security

Practice	How
Dependabot / Renovate	Not configured. **Recommendation:** enable Dependabot security alerts in GitHub Settings → Security
`pnpm audit`	Run periodically. Won't catch everything but catches known CVEs in direct deps
Pin major versions	`package.json` uses `^` (caret) — patches and minor updates auto-pick up on `pnpm install`. Risky for surprise breaking changes; acceptable for security patches

---

XSS / injection considerations

Vector	Status
User-provided strings rendered in HTML	All user input goes through React, which escapes by default. The contact form's `message` and gift card `note` are explicitly `escapeHtml`'d in email templates (`lib/email/contact-template.ts`, `lib/email/gift-card-template.ts`).
GROQ injection	Sanity queries use parameterized `$param` syntax — never string-concatenate user input into GROQ.
SQL injection	N/A — no SQL database. Sanity is the data store.
Open redirects	The success page reads `payment_intent` from the URL but only uses it to look up Stripe metadata; it's not used for navigation. No `?redirect=` parameters anywhere.

Email security

- **Domain auth (SPF/DKIM/DMARC):** Resend Dashboard → Domains → `stoneiwc.com` should show all green
- `replyTo` is set on every transactional email to `info@stoneiwc.com` — replying lands in the team inbox, not the no-reply void
- **Subject lines** avoid spam triggers ($ amounts, ALL CAPS, excessive punctuation)
- **No customer email lists** stored — the contact form sends one-off messages only, no newsletter or marketing mail

---

Compliance posture

This is a small business e-commerce site. We are NOT:

- GDPR-certified (no formal DPA, no data processor agreements)
- SOC 2 / ISO 27001 audited
- HIPAA-covered (no health data despite the "Wellness Center" name — services are educational/lifestyle, not medical)
- PCI-DSS audited (we're out of PCI scope by virtue of using Stripe Elements)

Customers in regulated jurisdictions (EU, California, etc.) have implicit rights under GDPR / CCPA. Honor deletion requests as described in Customer PII above. Add a privacy policy and cookie banner if not already present.

"If something looks wrong" — what to do

Signal	Action
Stripe Dashboard shows charges you didn't authorize	Rotate `STRIPE_SECRET_KEY` immediately, audit access
Sanity has order docs you didn't create	Rotate `SANITY_API_TOKEN`, audit Sanity Manage → Activity log
Emails sent from `info@stoneiwc.com` that you didn't send	Rotate `RESEND_API_KEY`, check Resend Logs for the offending sends
Webhook handler running with unexpected events	Check Stripe → Webhooks → Endpoint configuration; remove any unauthorized endpoints
Customer reports their account was used without permission	We have no accounts — confirm the customer isn't confusing us with another site. If they ordered with stolen card, refund and let Stripe's fraud detection handle the rest

Document any incident in `internalNotes` on the related order, plus a separate post-mortem to the team.

See also

- environments-and-deployments.md — env var management
- webhooks.md — webhook signature details
- runbook.md — incident response patterns
- open-backlog.md — server-side price validation, audit log, other hardening items

Open Backlog & Known Limitations

What's intentionally not built (yet), what's missing but minor, and what would be nice. Useful for a successor who needs to know "is this a bug or just a known gap?"

Grouped by area, ordered roughly by impact. Items here are NOT blocking production — the site works without them.

Storefront / orders

Abandoned cart cleanup

What: Pending order documents in Sanity stay forever if the customer never confirms payment.

Why open: Not enough volume yet to be a real problem. They're easy to filter (status = pending, older than 24h) and bulk-delete manually.

Future fix: A Vercel Cron job that runs daily, finds `pending` orders older than 24 hours, and deletes them. ~20 lines of code. Add to `app/api/cron/cleanup-pending-orders/route.ts`, wire up in `vercel.json`'s `crons` config.

Customer order history page

What: No `/account/orders` page. Customers can't see past purchases.

Why open: Would require user authentication, which we don't have. Adding auth is a meaningful project (Clerk / NextAuth / Sanity auth all possibilities).

Future fix: Add auth (Clerk recommended on Vercel — Marketplace integration). Then query Sanity by `customer.email` for the order list. Email confirmation already contains the order number for one-off lookup.

Server-side price re-validation

What: The server creates the Stripe PaymentIntent using the client-sent `totalAmount`. A determined attacker could send a low value via the API and pay less than the real price.

Mitigations today:

- Sanity is read-only client-side, so the prices shown in the UI are correct
- The server records the breakdown (subtotal, coupon, gift card, shipping) sent by the client into the order doc — divergence from the total is visible if you audit

Why open: No actual exploitation observed; impact is "discount" not "free", since the customer still pays *something*. Real fix is non-trivial because the cart/checkout flow trusts the client's computed total.

Future fix: In `create-stripe-payment-intent/route.tsx`, before creating the PI:

1. Re-fetch each item's price from Sanity by ID
2. Re-fetch the coupon by code (already done indirectly via `validate-coupon`)
3. Re-compute the expected total
4. Reject (or correct) if it disagrees with the client value by more than $0.01

Custom shipping notification email

What: When the order status flips to `shipped`, no email is sent. Customer has to check Sanity-managed tracking — which they don't have access to.

Why open: Time/priority — order confirmation was the higher-priority email.

Future fix: Send a Resend email from `lib/orders.ts` when status transitions to `shipped`. Template: `lib/email/order-shipped-template.ts`. Trigger: either a Sanity webhook (preferred — fires on the actual status change in Studio) or a polling cron.

Inventory / stock tracking

What: Products have `inStock: boolean` but no quantity. Selling out doesn't auto-flip the flag.

Why open: Not enough scale to warrant quantity tracking. Easy to set `inStock: false` manually.

Future fix: Add `quantity` field to Sanity product schema. Decrement on `markOrderPaid` (atomically per item, with concurrency guards similar to gift card redemption). Show "X left" badge under low-stock threshold.

Sanity validation: shipping cost minimum

What: Shipping `cost` field accepts any value ≥ 0. Setting it between $0 and $0.50 creates orders Stripe will reject (below minimum charge).

Why open: Trust-based — no admin has set this incorrectly so far.

Future fix: Add validation rule to `studio/schemaTypes/shippingMethod.ts` — `Rule.custom((cost) => cost === 0 || cost >= 0.5 || 'Cost must be 0 (free) or at least $0.50')`.

`payment_intent.canceled` not handled

What: Webhook subscribes to `succeeded` + `payment_failed`. The `canceled` event (when a PI is canceled before payment, e.g. via API) is ignored.

Why open: Not generated in normal customer flow. Only happens if an admin explicitly cancels a PI.

Future fix: Add `canceled` to the subscribed events on both Stripe webhook endpoints, add a handler branch that marks the order as `cancelled`.

Gift cards

No "resend gift card email" button

What: If a recipient loses the email or the original send bounced, there's no UI to resend. An admin has to email the code manually.

Future fix: Add a Studio action (Sanity has a "Document actions" API) that calls a new `/api/gift-cards/resend-email` endpoint.

No expiry warning emails

What: Gift cards default to 1-year expiry. We don't warn the recipient or purchaser as the date approaches.

Future fix: Vercel Cron job, daily query for cards expiring in 30 / 7 / 1 days, send reminder email.

No multi-currency

What: Gift cards are USD-only. So is the rest of the site.

Future fix: Not a near-term need — Stone IWC is US-based.

Bookings (Cal.com)

No record of bookings in Sanity

What: Bookings stay in Cal.com. The storefront has no view of who booked what.

Mitigation: Admins can use Cal.com directly for booking management.

Future fix: Subscribe to Cal.com webhooks, mirror bookings into Sanity similarly to how orders are mirrored.

Manual event provisioning

What: Adding bookable services requires either using Cal.com's UI or running the Python script in `scripts/`. No Studio integration.

Future fix: A Sanity-based service catalog that auto-syncs to Cal.com via API. Significant project — out of scope.

Email / Resend

Single email domain

What: All emails come from `info@stoneiwc.com`. No transactional / marketing separation.

Why open: Small enough scale that one domain works. Mixing transactional with marketing risks deliverability.

Future fix: Use a separate subdomain for marketing (`mail.stoneiwc.com`) with separate Resend domain auth. Transactional stays on `info@`.

No email click/open tracking

What: Resend supports it; we haven't enabled it.

Why open: Privacy + complexity tradeoff. Customer transactional emails don't need it.

SEO

Sanity webhook → revalidate not wired

What: When content is edited in Sanity Studio, ISR pages refresh on the next 60-second cycle. There's no instant revalidation.

Future fix: Configure Sanity Studio webhook to POST to `/api/revalidate` with a secret. Already supports it — see `app/api/revalidate/route.ts` for the entry point.

No automated sitemap submission

What: Sitemap is generated at `app/sitemap.ts` but not submitted to Google Search Console automatically.

Future fix: One-time submission via Search Console UI is enough; subsequent crawls find it via robots.txt. Doc this in seo.md "post-deploy steps".

No Schema.org structured data on every page

What: Product detail pages have JSON-LD via `components/seo/`, but `Organization`, `BreadcrumbList`, `FAQPage` etc. aren't everywhere they could be.

Future fix: Audit with Google Rich Results Test, add structured data where it boosts SERP appearance.

Frontend

No dark mode

What: CSS tokens for dark mode are defined but no toggle. Customers see only light.

Why open: Design choice — restrained, editorial look is light-first.

Future fix: Add a `<ThemeProvider>` consumer to layout, a toggle in the navbar/footer, use `next-themes` package.

No accessibility audit performed

What: No formal a11y review. Lighthouse score may be in the 80s.

Future fix: Run Lighthouse, fix top issues (typically contrast, missing labels). See frontend-ui.md.

No image optimization beyond Next.js defaults

What: Sanity images go through `urlFor()` with manual width/height. Could use responsive `srcset` more aggressively.

Future fix: Use `next/image` with `sizes` prop on every product/article image — it generates the responsive set automatically.

DevOps / observability

No structured error logging beyond `console.error`

What: Failures are logged with `console.error(...)` and end up in Vercel function logs. No error aggregation (Sentry, Datadog, etc.).

Future fix: Install Sentry (Vercel Marketplace one-click). Wrap critical paths (webhook, checkout) with error boundaries that report.

No CI / pre-merge checks

What: No GitHub Actions workflow. No PR builds beyond what Vercel does.

Future fix: A workflow that runs `pnpm lint` + `pnpm tsc --noEmit` + `pnpm build` on every PR. Block merge on failure.

No automated tests

What: No unit or integration tests in the repo.

Why open: Small codebase, integration-test-y by nature (Stripe + Sanity + Resend in a flow). Hard to mock realistically.

Future fix: Start with the high-value, contained logic — `lib/orders.ts`, `lib/gift-cards.ts`, the coupon calculation in `lib/cart-context.tsx`. Vitest is the obvious choice.

Vercel CLI is outdated locally

What: `vercel` CLI on developer machines is older than `vercel@latest`. Some commands miss newer features.

Future fix: `pnpm add -g vercel@latest` (or npm equivalent). Document in onboarding.md.

Security

`SANITY_API_TOKEN` shared across environments

What: Same token used for Prod, Preview, Dev. Compromise of dev exposes prod write access.

Future fix: Generate separate tokens per scope. Trade-off: more tokens to rotate.

Rate limiting is in-memory (resets on cold start)

What: Contact form rate limit lives in memory, dies on each function instance restart.

Future fix: Move to Vercel Edge Config or Upstash Redis for persistent rate limit state.

No audit log of admin actions

What: When an admin changes order status, gift card balance, etc., there's no log of who did what when. Sanity tracks revisions but doesn't surface them well in the UI.

Future fix: Either use Sanity's built-in history (already there), or add `lastModifiedBy` field to mutating documents.

Documentation

Schema docs lag behind code

What: Sanity schemas evolve; sanity-cms.md is updated manually.

Future fix: Auto-generate the schema reference from `studio/schemaTypes/*.ts` files. Or a pre-merge check that fails if schemas were touched without docs.

No video walkthrough for content workflows

What: content-workflows.md is text. Non-technical staff would benefit from a Loom or YouTube.

Future fix: Record 5-minute walkthroughs of common tasks. Link from the doc.

How to use this list

When picking up the project:

- Items here are **deliberate omissions** or **acknowledged debt** — not bugs
- The site works without any of them — they're prioritized "nice to haves"
- Some are quick wins (1–2 hour fix); others are projects of their own
- If you're adding a feature that touches one of these areas, see if it's worth fixing the related item at the same time

If you fix something here, delete the section.

If you discover a new limitation worth documenting, add it.